

claude-windows / 75d3d121-f240-4293-afa5-b436a379ad31

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\subagents\workflows\wf_5f13fc79-bad\agent-a588aba9828101736.jsonl`  
- SHA-256: `e0970c2d31884c0fa84150401fc61ac1e29dc639692a2508c8470ca6e4101404`  
- Source modified: `2026-07-08T06:16:23+00:00`  
- Imported at: `2026-07-08T15:59:27+00:00`  
- project: `wf_5f13fc79-bad`  
- session_id: `75d3d121-f240-4293-afa5-b436a379ad31`
```

Transcript

1. user (2026-07-08T06:04:07.732Z)

CONTEXT: This is GitHub issue #521 (Khelsutra/badminton-highlight-indexer), MERGED as PR #525. It makes pipeline phase?machine placement config-driven and dispatches the reel compile/stitch to an L4 Cloud Run worker (NVENC), mirroring the existing detection dispatch.

The full unified diff is at: undefined (read it first).

The full post-change source is under: undefined/ (read whole files there for context ? e.g. undefined/backend/orchestration.py, undefined/backend/worker/__main__.py, undefined/backend/compute/cloud_run.py, undefined/backend/config/placement.py, undefined/backend/config/models.py, undefined/backend/pipeline/stitcher/compiler.py).

Key design invariants that MUST hold (a violation is a real bug):

- DEFAULT-OFF: with no phase_placement config and compute_target unset, behaviour is byte-identical to before #521 (validation+compile on control-plane, detect in-process).
- The cloud compile path must return the SAME {status, filepath, download_url, video_id} shape the SPA already polls for; a stitch failure must be a clean failed-job, never a crash.
- The worker 'compile' subcommand reads a bucket-staged spec (resolved time-pairs + stitching config + source uri) and needs NO control-plane DB. It reuses orchestration._stitch_reel (shared core).
- Encoder: all clips in one reel must share ONE codec so the concat stays -c copy. NVENC is probed once and falls back to libx264 if unavailable (CI/non-GPU) ? the fallback must be all-or-nothing, never per-clip (which would break the concat).
- The config guard rejects validation=cloud_run_l4 without a cloud segment (nobody-validates footgun).
- Marker prefixes: compile uses _compile/, detect uses _dispatch/ ? must not collide.

Only report DEFECTS in the changed code (logic bugs, broken invariants, edge cases that produce a wrong result / crash / silent data loss, or missing test coverage of a real failure mode). Do NOT report style, naming, or speculative concerns. Cite exact file:line from undefined.

Your review dimension: parity-concurrency.

Review the #521 diff for PARITY + CONCURRENCY defects. Check: all clips share one encoder (no per-clip codec drift that breaks concat -c copy); the NVENC probe cache is correct; _LOCAL_COMPUTE_LANE usage; done-marker prefix collisions (_compile vs _dispatch); token uniqueness; any race between staging the spec and dispatch; the worker writing the reel to a

bucket vs local output_dir (reel_object_uri returning '' for a non-URI root).

2. assistant (2026-07-08T06:04:16.063Z)

I'll start by locating the relevant code. The diff/source paths came through as "undefined", so let me find the actual PR #525 changes in the repo.

3. user (2026-07-08T06:04:19.011Z)

bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells (\$9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload (#518)
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)
115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)
ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker (#515)
f5c9dd9 chore(ui): re-vendor SPA bundle with the hosted compute-picker cloud default (#516)
ec2d080 fix(serving): fail fast when compute=local forces weightless in-process detection (#511)
20862d7 perf(validators): decode scene_cut/blur/relevance frames in one shared pass (#512)
23c1868 fix(serving): worker skips redundant re-validation on the cloud dispatch path (#513)
3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image (#510)
66756ad chore(serving): forward WASB decode/tuning knobs to worker + release hardening (#509)
143dbb9 feat(serving): default cloud-serving to nvdec_resident decode (#508)
44a2f47 feat(detectors): GPU-resident WASB NVDEC decode behind default-OFF decode_backend flag (#507)
17d844b docs(runlog): alpha-tester "All 1 v 1" cloud-serving run ? timeout diagnosed + downsample fix proven (#504)
bd5647b fix(worker): surface detector failure/timeout as failed, not silent 0-rally success (#503)

4. user (2026-07-08T06:04:20.101Z)

dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement (#521) (#525)
8866ffe perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement (#521)
a5d6246 serving: re-queue interrupted compile jobs on restart (#329 reliability) (#460)
b09a1d4 serving: re-queue interrupted compile jobs on restart (#329 reliability)
d5a1fea fix: Phase 2 ? Q7 (motion type-clean) + B2 (Gemini error-flagging) + D4 (deep config merge) (#373)
947a36b docs: update CODE_CLEANUP_AUDIT ? Phase 1 complete, Phase 2 prioritized (#372)
739d1f4 fix: Q2 motion return type + D11 compiler codec docstring (#362)
bab72c6 docs: update cleanup project tracker ? Phase 1 items shipped (#357)
8e3aff1 docs: 2026-06-24 code-cleanup audit ? 33-item project doc (Phase 1 approval gate) (#352)
8bcf6ce feat(api): make /api/compile async (202 + poll) to kill the slow-stitch 524 (#329) (#339)
e221cb9 docs(data): GCS corpus source-of-truth map + Phase-1 metadata migration (#316)
e99d0c3 feat(stitch): localize the reel watermark to the UI locale (+ OFL Noto fonts) (#304)
e68b583 fix(api): resolve a bucket-ingested video's source to a local file before stitching (#289)
e6110ce feat(api): GET /api/health + return download_url from /api/compile (M1/P6) (#268)
dd30e9b feat(workspace): per-video workspace convention ? design doc + Phase 0 helper (#264)
8092a39 refactor(segmenter): decompose yolo_hybrid process_video into phase helpers (#227)
18883b5 refactor(stitcher): extract per-segment trim into _trim_one_segment (#224)
8736e2d fix(runner): honor stitching watermark/padding + add --detect-from for hi-res reels (#205)
562308b fix(stitcher): merge overlapping/duplicate segments so a reel never stitches the same rally twice (#180)
428ee44 address follow-up review on #169: correct factual error in item-3 investigation note (fusion_compare.py, fusion_golden.py, ablation.py + tests *do* exist per ls/grep on the tree; person features in fusion_hybrid + detectors/fusion_features.py; remaining M0 in-container task is wiring into compare). Cleaned duplicate sequencing paragraph. Verified files present. Plan

now accurate. M0 item 3 placement remains correct and even stronger.
 318f6c2 test: positive tests for Item 2 (config) - shallow section replacement, unknown keys
 warning, get shim, roundtrip + living doc update for T2 post #157
 8439c24 feat(personalization): per-user store (v4/v5) + serving bridge groundwork (Phase 1, additive) (#150)
 e75ec35 feat(eval): LOS0/LOG0 ablation runner ? per-signal marginal ?F1 (Phase 0) (#147)
 7c6498f docs(quality): personalization Phase-0 checkpoint ? eval/safety rails + locked structural-guard decision (#145)
 8836dfe chore: scripts/ for run-drivers + CODE_MAP \$0 file-placement guidelines (#138)
 6de70aa fix(stitcher): escape drive-letter colon in ffmpeg filtergraph paths (#108)
 3461898 feat(api): enforce stitching.min_shot_count on /api/compile (#107)
 112e75d feat(stitcher): bundle Apache-2.0 default watermark font + log fallback reason
 82e479f Merge pull request #102 from avidullu:fix/stitching-padding-config
 30e4899 fix(api): thread config stitching paddings into live VideoCompiler sites

5. assistant (2026-07-08T06:04:27.077Z)

PR #525 landed as commit `dac6736` (with `8866ffe`). Let me pull the full diff and the current source of the key files.

6. user (2026-07-08T06:04:28.818Z)

commit dac6736cb23a681135b707d768014820b239a981
 Author: Avi <avi.dullu@gmail.com>
 Date: Wed Jul 8 11:29:33 2026 +0530

perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement (#521) (#525)

The compile/stitch phase ran in-process on the CPU-only 2-vCPU control-plane; the 2026-07-08 live CUJ (rev rally-control-plane-00019, §9 of the validation-latency runlog) measured ~12m55s for an 11-clip / ~80s reel from a 1.83 GiB 4K source (~45-90s per ~7s clip ? the per-clip branding watermark forces a full 4K re-encode with no NVENC). Validation (183s) also runs there; both serialize on _LOCAL_COMPUTE_LANE, so one run pins the control-plane ~16 min ? won't scale past a handful of concurrent users.

- Config-driven placement: deployment.phase_placement {validation|segment|compile} ? control_plane | cloud_run_l4, resolved by backend/config/placement.py. Unset ? today's placement, byte-identical. A config-load guard rejects the "nobody validates" footgun (validation on L4 without a cloud segment).
- compile ? L4: a kind='compile' Cloud Run dispatch mirroring _maybe_dispatch_remote. The control-plane resolves the reel's segments, stages a compile-spec (resolved time-pairs + stitching config + source uri) to the bucket, dispatches `worker compile` on the L4, bucket-polls a done-marker, and returns the same {status, filepath, download_url} shape the SPA expects. The worker stitches via the SHARED _stitch_reel core (no forked pipeline) and needs no control-plane DB.
- NVENC: StitchingConfig.encoder + deployment.cloud_stitch_encoder (default h264_nvenc) forwarded to the L4; VideoCompiler probes it once and falls back to libx264 (CPU) everywhere else. Lands the D16(b)-deferred NVENC-behind-a-config-knob now that the CUJ supplies the cost data; libx264 stays the parity-safe default.
- validation delegation: phase_placement.validation moves the validator suite off the control-plane via the existing --skip-validation lever (#513), guarded so an in-process run always validates.
- cloud-serving preset places compile on the L4.

Docs: docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md (+ README pointer).
 Tests: new test_phase_placement.py, test_compile_dispatch.py; additions to test_compute_backend.py (run_compile), test_worker.py (cmd_compile), test_compiler.py (encoder selection), test_api_cloud_dispatch.py (validation delegation). Full suite green (1875 passed), ruff/mypy clean, coverage 85.7%.

Co-authored-by: Claude Opus 4.8 <noreply@anthropic.com>

backend/compute/base.py	34 +++
backend/compute/cloud_run.py	100 +++++-
backend/config/__init__.py	16 ++
backend/config/models.py	52 ++++
backend/config/placement.py	59 ++++
backend/config/presets.py	10 +-
backend/orchestration.py	322 ++++++-----
backend/pipeline/stitcher/compiler.py	94 +++++-
backend/worker/__main__.py	139 ++++++
docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md	169 ++++++
docs/COMPUTE_DECOUPLED_SERVING/README.md	1 +
tests/test_api_cloud_dispatch.py	70 +++++
tests/test_compile_dispatch.py	204 ++++++
tests/test_compiler.py	64 +++++
tests/test_compute_backend.py	111 ++++++
tests/test_config_deployment.py	5 +
tests/test_phase_placement.py	138 ++++++
tests/test_worker.py	175 ++++++

7. user (2026-07-08T06:04:29.919Z)

```

dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
($9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload
(#518)
---PARENTS---
dac6736cb23a681135b707d768014820b239a981 bf97700facc2c7762a4cb2c7c039e7673eebab8b

```

8. assistant (2026-07-08T06:04:33.922Z)

This is a squash merge (single parent). Let me get the full diff for the key files, focusing on parity-concurrency surfaces.

9. user (2026-07-08T06:04:35.661Z)

```

commit dac6736cb23a681135b707d768014820b239a981
Author: Avi <avi.dullu@gmail.com>
Date: Wed Jul 8 11:29:33 2026 +0530

```

```

perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement
(#521) (#525)

```

The compile/stitch phase ran in-process on the CPU-only 2-vCPU control-plane; the 2026-07-08 live CUJ (rev rally-control-plane-00019, \$9 of the validation-latency runlog) measured ~12m55s for an 11-clip / ~80s reel from a 1.83 GiB 4K source (~45-90s per ~7s clip ? the per-clip branding watermark forces a full 4K re-encode with no NVENC). Validation (183s) also runs there; both serialize on _LOCAL_COMPUTE_LANE, so one run pins the control-plane ~16 min ? won't scale past a handful of concurrent users.

- Config-driven placement: deployment.phase_placement {validation|segment|compile} ? control_plane | cloud_run_l4, resolved by backend/config/placement.py. Unset ? today's placement, byte-identical. A config-load guard rejects the "nobody validates" footgun (validation on L4 without a cloud segment).
- compile ? L4: a kind='compile' Cloud Run dispatch mirroring _maybe_dispatch_remote. The control-plane resolves the reel's segments, stages a compile-spec (resolved time-pairs + stitching config + source uri) to the bucket, dispatches `worker compile` on the L4, bucket-polls a done-marker, and returns the same {status, filepath, download_url} shape the SPA expects. The worker stitches via the SHARED _stitch_reel core (no forked pipeline) and needs no control-plane DB.
- NVENC: StitchingConfig.encoder + deployment.cloud_stitch_encoder (default h264_nvenc) forwarded to the L4; VideoCompiler probes it once and falls back to

- libx264 (CPU) everywhere else. Lands the D16(b)-deferred NVENC-behind-a-config-knob now that the CUJ supplies the cost data; libx264 stays the parity-safe default.
- validation delegation: phase_placement.validation moves the validator suite off the control-plane via the existing --skip-validation lever (#513), guarded so an in-process run always validates.
- cloud-serving preset places compile on the L4.

Docs: docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md (+ README pointer).

Tests: new test_phase_placement.py, test_compile_dispatch.py; additions to test_compute_backend.py (run_compile), test_worker.py (cmd_compile), test_compiler.py (encoder selection), test_api_cloud_dispatch.py (validation delegation). Full suite green (1875 passed), ruff/mypy clean, coverage 85.7%.

Co-authored-by: Claude Opus 4.8 <noreply@anthropic.com>

```
diff --git a/backend/compute/base.py b/backend/compute/base.py
index 3ad76fc..f7afb40 100644
--- a/backend/compute/base.py
+++ b/backend/compute/base.py
@@ -51,6 +51,24 @@ class ComputeJobSpec:
     skip_validation: bool = False

+@dataclass(frozen=True)
+class CompileJobSpec:
+    """One reel-COMPILE job dispatched to the worker (#521). The heavy input ? the resolved
+    ``(start, end)`` time-pairs plus the control-plane's stitching/watermark config and the
+    source
+    URI ? is staged to ``spec_uri`` (a small JSON in the bucket) rather than crammed onto argv,
+    so
+    the worker needs no control-plane DB: it reads the spec, fetches the source, stitches on
+    the L4
+    (NVENC), uploads the reel to ``<out_uri>/highlights/<video_id>/<output_name>``, and writes
+    a
+    done-marker for the dispatcher's bucket-poll."""
+    out_uri: str # output ROOT (gs://? bucket); highlights/<video_id>/<name> is written under
+    it
+    spec_uri: str # staged compile-spec JSON (time-pairs + stitching + source uri + video_id)
+    video_id: str # the reel's video_id (deterministic reel path + marker)
+    output_name: str # traversal-safe reel filename
+    # Raw worker ``--set k=v`` overrides (storage.output_root/backend + stitching.encoder). A
+    tuple
+    # (not list/dict) so the spec stays frozen/hashable.
+    config_overrides: tuple[str, ...] = ()
+
+@dataclass(frozen=True)
+class ComputeResult:
+    """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit code
@@ -85,3 +103,19 @@ class ComputeBackend(ABC):
     ``on_wait(elapsed_sec)`` (optional) is invoked periodically while the backend is
     still WAITING on the remote work ? the caller's live-progress heartbeat (#482), so a
     long GPU run doesn't look frozen. Implementations without a wait loop may ignore it."""
+
+    def run_compile(
+        self,
+        spec: "CompileJobSpec",
+        *,
+        on_wait: "Callable[[float], None] | None" = None,
+    ) -> ComputeResult:
+        """Run a reel-COMPILE ``spec`` to completion and return its ``ComputeResult`` (#521).
+
+        Non-abstract with a NotImplementedError default: only backends that can offload the
+        ffmpeg
```

```

+         stitch (`CloudRunCompute`) override it ? an in-process/other backend never dispatches
a
+         compile. The returned ``ComputeResult.returncode`` is the worker's rc (0 ok / non-zero
=
+         stitch failure) and ``report`` is the worker's done-marker (carries the reel's
download_url)."""
+         raise NotImplementedError(
+             f"{type(self).__name__} does not support remote compile dispatch"
+         )
diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
index a4a0743..8df4246 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -37,6 +37,7 @@ from abc import ABC, abstractmethod
 from typing import Any, Callable, List, Optional

 from backend.compute.base import (
+
+     CompileJobSpec,
+     ComputeBackend,
+     ComputeJobSpec,
+     ComputeResult,
@@ -278,21 +279,7 @@ class CloudRunCompute(ComputeBackend):
     done_uri,
    )

-
-    try:
-        marker = poll_until(
-            lambda: self._poll_check(done_uri, str(execution or "")),
-            self._policy,
-            label="cloud-run-infer",
-            logger=LOG,
-            # Live heartbeat (#482): each still-waiting tick reaches the caller so
-            # job.progress moves during the whole GPU run instead of freezing.
-            on_tick=on_wait,
-        )
-    except PolicyTimeout as timeout_exc:
-        # RemoteExecutionFailed (the fast negative path in _poll_check) is NOT a
PolicyTimeout,
-        # so it propagates past here uncaught ? the dispatch handler maps it to a distinct,
-        # clearly-labelled failure instead of the ~65-min opaque timeout this branch
handles.
-        marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
+        marker = self._await_marker(done_uri, execution, on_wait, label="cloud-run-infer")
+        video_id = str(marker.get("video_id", ""))
+        # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
+        # garbled marker must not masquerade as rc=0. (The worker always writes both fields.)
@@ -317,6 +304,89 @@ class CloudRunCompute(ComputeBackend):
     execution=str(execution or ""),
    )

+
+    def run_compile(
+        self,
+        spec: "CompileJobSpec",
+        *,
+        on_wait: "Callable[[float], None] | None" = None,
+    ) -> ComputeResult:
+        """Dispatch a reel-COMPILE (#521): run ``python -m backend.worker compile`` on the L4
(NVENC),
+        bucket-poll the done-marker, and return the worker's rc + marker. Reuses the EXACT
+        dispatch?poll?marker machinery as ``run_infer`` (``_await_marker`` ? crash-fast +
timeout
+        rescue), so a stitch that crashes or overruns fails the same way a detection does ? not
a
+        ~65-min opaque hang."""

```

```

+         out_root = spec.out_uri.rstrip("/")
+         token = self._token or uuid.uuid4().hex
+         # Distinct marker prefix from infer's _dispatch/ so a compile + a detection for the
same
+         # bucket can never collide on a marker.
+         done_uri = f"{out_root}/_compile/{token}.done"
+         argv: List[str] = [
+             "compile",
+             "--out-uri",
+             spec.out_uri,
+             "--spec-uri",
+             spec.spec_uri,
+             "--done-uri",
+             done_uri,
+         ]
+         # Compile carries NO container-inline overrides: cmd_compile never consults
compute_target and
+         # never re-dispatches, so the only overrides are the CP-forwarded storage/encoder
knobs.
+         for kv in spec.config_overrides:
+             argv += ["--set", kv]
+
+         execution = self._client.run_job(
+             self._job_name, argv, region=self._region, project=self._project
+         )
+         LOG.info(
+             "cloud-run: dispatched COMPILE job=%s exec=%s; bucket-polling %s",
+             self._job_name,
+             execution,
+             done_uri,
+         )
+         marker = self._await_marker(done_uri, execution, on_wait, label="cloud-run-compile")
+         video_id = str(marker.get("video_id", "") or spec.video_id)
+         rc = int(marker.get("returncode", 1))
+         LOG.info(
+             "cloud-run: COMPILE job=%s done (video_id=%s rc=%d)",
+             self._job_name,
+             video_id,
+             rc,
+         )
+         # The marker carries the worker's stitch result (download_url / reel_uri / status);
surface it
+         # as ``report`` so the dispatcher returns the same {status, filepath, download_url}
shape the
+         # SPA expects. ``outputs_uri`` is the mirrored reel URI when present.
+         return ComputeResult(
+             returncode=rc,
+             outputs_uri=str(marker.get("reel_uri", "") or ""),
+             video_id=video_id,
+             report=marker,
+             execution=str(execution or ""),
+         )
+
+     def _await_marker(
+         self,
+         done_uri: str,
+         execution: object,
+         on_wait: "Callable[[float], None] | None",
+         *,
+         label: str,
+     ) -> dict:
+         """Bucket-poll ``done_uri`` to a completion marker ? shared by ``run_infer`` and
+         ``run_compile``. ``on_wait`` is the #482 live heartbeat (each still-waiting tick
reaches the
+         caller so job.progress moves). A ``RemoteExecutionFailed`` (the fast negative path in

```

```

+         ``_poll_check``) is NOT a ``PolicyTimeout`` so it propagates uncaught ? the dispatch
handler
+         maps it to a distinct failure; a genuine poll TIMEOUT is handled (rescue-or-cancel)."""
+         try:
+             return poll_until(
+                 lambda: self._poll_check(done_uri, str(execution or "")),
+                 self._policy,
+                 label=label,
+                 logger=LOG,
+                 on_tick=on_wait,
+             )
+         except PolicyTimeout as timeout_exc:
+             return self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
+
+     def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
+         """One bucket-poll iteration, hardened with an Executions-API fast negative path.

diff --git a/backend/pipeline/stitcher/compiler.py b/backend/pipeline/stitcher/compiler.py
index 65d1c57..5c7229e 100644
--- a/backend/pipeline/stitcher/compiler.py
+++ b/backend/pipeline/stitcher/compiler.py
@@ -23,6 +23,49 @@ _BUNDLED_FONT = (
    Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-Regular.ttf"
)

+ #: Per-encoder availability memo (#521): probing ffmpeg is a subprocess, so a run's first clip
+ #: pays
+ #: it and the rest reuse the verdict. Keyed by encoder name; cleared only across processes.
+ _ENCODER_AVAILABLE: Dict[str, bool] = {}
+
+
+ def _probe_encoder_available(encoder: str) -> bool:
+     """Can this ffmpeg build actually RUN ``encoder`` (not merely list it)? Runs a tiny 1-frame
+     lavfi test encode to null. NVENC is often *compiled in* on boxes without an NVIDIA GPU, so
+     a
+     real test encode ? not ``-encoders`` grep ? is what distinguishes "usable" from "will fail
+ at
+ runtime". Memoized. Any error / timeout ? unavailable (caller falls back to libx264)."""
+     if encoder in _ENCODER_AVAILABLE:
+         return _ENCODER_AVAILABLE[encoder]
+     ok = False
+     # The probe is a tiny 1-frame encode ? bound it to a short timeout (? the configured ffmpeg
+     # budget, or 30 s when unbounded) so a wedged encoder can't hang the whole stitch.
+     configured = ffmpeg_timeout_sec()
+     probe_timeout = min(30.0, float(configured)) if configured else 30.0
+     try:
+         proc = subprocess.run(
+             [
+                 ffmpeg_bin(),
+                 "-hide_banner",
+                 "-f",
+                 "lavfi",
+                 "-i",
+                 "color=c=black:s=64x64:d=0.1",
+                 "-frames:v",
+                 "1",
+                 "-c:v",
+                 encoder,
+                 "-f",
+                 "null",
+                 "-",
+             ],
+             capture_output=True,
+             timeout=probe_timeout,
+         )

```



```

+         ok = proc.returncode == 0
+     except Exception: # noqa: BLE001 ? a probe failure/timeout just means "not usable ? fall
back"
+         ok = False
+     _ENCODER_AVAILABLE[encoder] = ok
+     return ok
+
class VideoCompiler:
    def __init__(
@@ -31,6 +74,7 @@ class VideoCompiler:
        end_padding: float = 1.0,
        begin_padding: float = 0.5,
        watermark: Optional[Any] = None,
+    encoder: str = "libx264",
    ):
        self.output_dir = output_dir
        self.end_padding = end_padding
@@ -38,6 +82,11 @@ class VideoCompiler:
        # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
        # normalized to a dict (or None) so this module stays config-package-agnostic.
        self.watermark = self._normalize_watermark(watermark)
+    # Video encoder for the per-clip trim re-encode (#521). "libx264" (CPU) is the default
and
+    # works everywhere; "h264_nvenc"/"hevc_nvenc" use the L4's HARDWARE encoder for the 4K
+    # re-encode the watermark forces. The requested encoder is PROBED once per run
+    # (_select_encoder) and falls back to libx264 if this ffmpeg build can't run it.
+    self.encoder = (encoder or "libx264").strip() or "libx264"
    os.makedirs(output_dir, exist_ok=True)

    @staticmethod
@@ -225,6 +274,30 @@ class VideoCompiler:
        return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
    return drawtext

+    def _select_encoder(self) -> Tuple[List[str], str]:
+        """Resolve the ``-c:v ?`` args used for EVERY per-clip trim re-encode this run, plus
the
+        chosen encoder name. Chosen ONCE (not per clip) so all clips share one codec and the
final
+        concat can stay ``-c copy``. NVENC (hardware) is the L4 win on the 4K re-encode; if the
requested NVENC encoder can't run in this ffmpeg build (CI / non-GPU box), fall back to
libx264 so a misconfig never nukes the reel."""
+        enc = self.encoder
+        if enc in ("h264_nvenc", "hevc_nvenc") and not _probe_encoder_available(enc):
+            logger.warning(
+                "[compile] encoder %r is not runnable in this ffmpeg build ? falling back to "
+                "libx264 (CPU). On the L4 worker this should be available.",
+                enc,
+            )
+            enc = "libx264"
+        if enc == "libx264":
+            # Unchanged historical CPU settings (byte-compatible with the pre-#521 stitch).
+            return ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"], enc
+        # NVENC (H.264/HEVC): hardware encode. -cq is the CRF-equivalent constant-quality knob;
p4 is
+        # a balanced speed/quality preset. yuv420p keeps the output broadly playable.
+        return (
+            ["-c:v", enc, "-preset", "p4", "-cq", "23", "-pix_fmt", "yuv420p"],
+            enc,
+        )

    def _trim_one_segment(
        self,
        idx: int,

```

```

@@ -235,6 +308,7 @@ class VideoCompiler:
    tmp_dir: str,
    raw_video_path: str,
    watermark_filter: Optional[str],
+    video_codec_args: Optional[List[str]] = None,
) -> Tuple[Optional[str], Optional[Tuple[int, float, float, str]]]:
    """Trim a single segment into its own clip file (extracted verbatim from the
    per-segment body of compile_highlights). Returns (clip_path, skip_record): on
@@ -297,6 +371,13 @@ class VideoCompiler:
    f"Trimming {segment_label} -> [{padded_start:.3f}s - {padded_end:.3f}s] (duration:
    {trim_duration:.2f}s)"
    )

+    # Default to libx264 (the historical settings) when a caller invokes _trim_one_segment
+    # directly without resolving an encoder ? keeps this method's old behaviour intact.
+    codec_args = (
+        video_codec_args
+        if video_codec_args is not None
+        else ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"]
+    )
+    base_cmd = [
+        ffmpeg_bin(),
+        "-y",
@@ -306,12 +387,7 @@ class VideoCompiler:
    str(round(padded_end, 3)),
    "-i",
    raw_video_path,
-    "-c:v",
-    "libx264",
-    "-preset",
-    "superfast",
-    "-crf",
-    "22",
+    *codec_args,
+    "-c:a",
+    "aac",
+    "-b:a",
@@ -445,6 +521,11 @@ class VideoCompiler:
    watermark_filter = self._watermark_filter(tmp_dir)
    if watermark_filter:
        logger.info("[watermark] applying branding overlay to each clip.")
+    # Resolve the video encoder ONCE (probe NVENC / fall back) so every clip shares one
+    codec
+    # and the final concat stays -c copy (#521).
+    video_codec_args, chosen_encoder = self._select_encoder()
+    if chosen_encoder != "libx264":
+        logger.info("[compile] encoding clips with %s (hardware).", chosen_encoder)
+    for idx, (raw_start, raw_end) in enumerate(segments):
+        clip_path, skip_record = self._trim_one_segment(
+            idx,
@@ -455,6 +536,7 @@ class VideoCompiler:
    tmp_dir,
    raw_video_path,
    watermark_filter,
+    video_codec_args,
    )
    if clip_path is not None:
        temp_clips.append(clip_path)

```

10. assistant (2026-07-08T06:04:40.036Z)

Now the worker and orchestration diffs.

11. user (2026-07-08T06:04:41.214Z)

commit dac6736cb23a681135b707d768014820b239a981

Author: Avi <avi.dullu@gmail.com>

Date: Wed Jul 8 11:29:33 2026 +0530

perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement (#521) (#525)

The compile/stitch phase ran in-process on the CPU-only 2-vCPU control-plane; the 2026-07-08 live CUJ (rev rally-control-plane-00019, §9 of the validation-latency runlog) measured ~12m55s for an 11-clip / ~80s reel from a 1.83 GiB 4K source (~45-90s per ~7s clip ? the per-clip branding watermark forces a full 4K re-encode with no NVENC). Validation (183s) also runs there; both serialize on `_LOCAL_COMPUTE_LANE`, so one run pins the control-plane ~16 min ? won't scale past a handful of concurrent users.

- Config-driven placement: `deployment.phase_placement {validation|segment|compile} ? control_plane | cloud_run_l4`, resolved by `backend/config/placement.py`. Unset ? today's placement, byte-identical. A config-load guard rejects the "nobody validates" footgun (validation on L4 without a cloud segment).
- compile ? L4: a kind='compile' Cloud Run dispatch mirroring `_maybe_dispatch_remote`. The control-plane resolves the reel's segments, stages a compile-spec (resolved time-pairs + stitching config + source uri) to the bucket, dispatches ``worker compile`` on the L4, bucket-polls a done-marker, and returns the same `{status, filepath, download_url}` shape the SPA expects. The worker stitches via the `SHARED _stitch_reel` core (no forked pipeline) and needs no control-plane DB.
- NVENC: `StitchingConfig.encoder + deployment.cloud_stitch_encoder` (default `h264_nvenc`) forwarded to the L4; `VideoCompiler` probes it once and falls back to `libx264` (CPU) everywhere else. Lands the `D16(b)-deferred NVENC-behind-a-config-knob` now that the CUJ supplies the cost data; `libx264` stays the parity-safe default.
- validation delegation: `phase_placement.validation` moves the validator suite off the control-plane via the existing `--skip-validation` lever (#513), guarded so an in-process run always validates.
- cloud-serving preset places compile on the L4.

Docs: `docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md` (+ README pointer).

Tests: new `test_phase_placement.py`, `test_compile_dispatch.py`; additions to `test_compute_backend.py` (`run_compile`), `test_worker.py` (`cmd_compile`), `test_compiler.py` (encoder selection), `test_api_cloud_dispatch.py` (validation delegation). Full suite green (1875 passed), ruff/mypy clean, coverage 85.7%.

Co-authored-by: Claude Opus 4.8 <noreply@anthropic.com>

```
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
```

```
index 6d3420b..79db808 100644
```

```
--- a/backend/worker/__main__.py
```

```
+++ b/backend/worker/__main__.py
```

```
@@ -419,6 +419,108 @@ def cmd_infer(args: argparse.Namespace) -> int:
```

```
    return 0
```

```
+def _read_compile_spec(spec_uri: str, storage_cfg: dict, scratch: str) -> dict:
```

```
+    """Fetch + parse the control-plane-staged compile-spec (#521): the resolved ``time_pairs``,  
+    the
```

```
+    resolved ``stitching`` config, the source ``video_uri``, ``video_id``, ``output_name`` +  
+    ``locale``.
```

```
+    A gs://|s3:// URI is downloaded to scratch first (cloud backends require a real dst)."""
```

```
+    local = storage.fetch(
```

```
+        spec_uri, os.path.join(scratch, "compile_spec.json"), config=storage_cfg
```

```
+    )
```

```
+    with open(local, encoding="utf-8") as f:
```

```
+        data = json.load(f)
```

```
+    if not isinstance(data, dict):
```

```
+        raise ValueError(f"compile spec at {spec_uri} is not a JSON object")
```

```
+    return data
```

```
+
```

```

+
+def _write_compile_marker(
+    done_uri: str,
+    result: dict,
+    video_id: str,
+    returncode: int,
+    storage_cfg: dict,
+    scratch: str,
+) -> None:
+    """#521: write the compile done-marker (rc + status + the reel's download_url/reel_uri) so
the
+    dispatcher's bucket-poll terminates and can return the SPA result. Best-effort ? never
raises."""
+    try:
+        marker = {
+            "video_id": video_id,
+            "returncode": returncode,
+            "status": result.get("status", "failed"),
+            "download_url": result.get("download_url", ""),
+            "reel_uri": result.get("reel_uri", ""),
+            "message": result.get("message", ""),
+        }
+        local = os.path.join(scratch, "_compile_done.json")
+        with open(local, "w", encoding="utf-8") as f:
+            json.dump(marker, f)
+        storage.put(local, done_uri, config=storage_cfg)
+        logger.info("[worker] wrote compile marker -> %s", done_uri)
+    except Exception as e: # never fail a good stitch because the marker push hiccuped
+        logger.warning("[worker] could not write compile done-marker %s: %s", done_uri, e)
+
+
+def cmd_compile(args: argparse.Namespace) -> int:
+    """#521: run the reel COMPILE/stitch on THIS worker (the L4 has NVENC + 8 vCPU). The stitch
is the
+    2026-07-08 CUJ's dominant term (~13 min on the 2-vCPU control-plane for an 11-clip 4K
reel). Read
+    the control-plane-staged compile-spec (resolved time-pairs + stitching config + source
URI), stitch
+    the reel via the SAME ``orchestration._stitch_reel`` core the control-plane uses (so there
is no
+    forked stitch pipeline), upload the reel to the bucket, and write a done-marker for the
dispatcher's
+    bucket-poll. No control-plane DB is needed ? the spec carries everything the stitch
requires."""
+    from backend import orchestration
+    from backend.config import StitchingConfig
+
+    config = load_config(args.config) if args.config else load_config()
+    _apply_overrides(config, args.set)
+    storage_cfg = config.storage.model_dump()
+
+    scratch = args.scratch or tempfile.mkdtemp(prefix="rally-compile-")
+    os.makedirs(scratch, exist_ok=True)
+    # Clips + the assembled reel land in scratch; the reel is then mirror-uploaded to the
bucket.
+    config.output.output_dir = scratch
+    # The dispatcher forwards storage.output_root via --set; --out-uri carries the same value
as a
+    # fallback so reel_object_uri (which reads storage.output_root) always resolves the reel's
bucket
+    # destination even if the override were dropped.
+    if not (config.storage.output_root or "").strip():
+        config.storage.output_root = args.out_uri
+
+    spec = _read_compile_spec(args.spec_uri, storage_cfg, scratch)

```

```

+ video_id = str(spec.get("video_id") or "")
+ out_name = str(spec.get("output_name") or "")
+ source_ref = str(spec.get("video_uri") or "")
+ locale = spec.get("locale")
+ time_pairs = [
+     (float(pair[0]), float(pair[1])) for pair in (spec.get("time_pairs") or [])
+ ]
+ stitching = StitchingConfig(**(spec.get("stitching") or {}))
+
+ print(
+     f"[worker] compile: video_id={video_id} clips={len(time_pairs)} "
+     f"encoder={stitching.encoder} out={out_name}"
+ )
+
+ result = orchestration._stitch_reel(
+     config,
+     stitching,
+     video_id,
+     source_ref,
+     time_pairs,
+     out_name,
+     locale,
+     lambda stage: print(f"[worker] compile: {stage}"),
+ )
+ status = str(result.get("status") or "failed")
+ rc = 0 if status == "success" else 3
+ if status != "success":
+     print(f"[worker] compile FAILED: {result.get('message')}")
+
+ # DEFAULT-OFF: no --done-uri ? no marker (a direct-CLI stitch). A remote dispatcher passes
it.
+ if getattr(args, "done_uri", None):
+     _write_compile_marker(args.done_uri, result, video_id, rc, storage_cfg, scratch)
+ return rc
+
+ #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter so a
+ #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem collision).
+ _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
@@ -760,6 +862,43 @@ def build_parser() -> argparse.ArgumentParser:
+ )
+ pi.set_defaults(func=cmd_infer)
+
+ pc = sub.add_parser(
+     "compile",
+     help="#521: stitch a highlight reel on this worker (L4 NVENC) from a staged compile-
spec",
+ )
+ pc.add_argument(
+     "--spec-uri",
+     required=True,
+     help="storage URI of the control-plane-staged compile-spec JSON (time-pairs + stitching
+
+     "config + source uri + video_id)",
+ )
+ pc.add_argument(
+     "--out-uri",
+     default="",
+     help="output ROOT (gs://? bucket); the reel is mirrored to
highlights/<video_id>/<name>. "
+     "Also forwarded via --set storage.output_root; either resolves the reel path.",
+ )
+ pc.add_argument(
+     "--done-uri",
+     default=None,

```

```

+         help="storage URI for the compile completion MARKER (rc + download_url), so a remote "
+         "dispatcher's bucket-poll terminates. Default-OFF: omit for a direct-CLI stitch.",
+     )
+     pc.add_argument(
+         "--config", default=None, help="config.json path (default: ./config.json)"
+     )
+     pc.add_argument(
+         "--scratch", default=None, help="local scratch dir (default: a temp dir)"
+     )
+     pc.add_argument(
+         "--set",
+         action="append",
+         default=[],
+         metavar="k=v",
+         help="config override, e.g. --set storage.output_root=gs://bucket",
+     )
+     pc.set_defaults(func=cmd_compile)
+
+     pt = sub.add_parser(
+         "train",
+         help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",

```

12. assistant (2026-07-08T06:04:45.473Z)

Now the orchestration diff, the largest surface.

13. user (2026-07-08T06:04:46.767Z)

```

commit dac6736cb23a681135b707d768014820b239a981
Author: Avi <avi.dullu@gmail.com>
Date:   Wed Jul 8 11:29:33 2026 +0530

```

perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement (#521) (#525)

The compile/stitch phase ran in-process on the CPU-only 2-vCPU control-plane; the 2026-07-08 live CUJ (rev rally-control-plane-00019, §9 of the validation-latency runlog) measured ~12m55s for an 11-clip / ~80s reel from a 1.83 GiB 4K source (~45-90s per ~7s clip ? the per-clip branding watermark forces a full 4K re-encode with no NVENC). Validation (183s) also runs there; both serialize on `_LOCAL_COMPUTE_LANE`, so one run pins the control-plane ~16 min ? won't scale past a handful of concurrent users.

- Config-driven placement: `deployment.phase_placement {validation|segment|compile} ? control_plane | cloud_run_l4`, resolved by `backend/config/placement.py`. Unset ? today's placement, byte-identical. A config-load guard rejects the "nobody validates" footgun (validation on L4 without a cloud segment).
- compile ? L4: a kind='compile' Cloud Run dispatch mirroring `_maybe_dispatch_remote`. The control-plane resolves the reel's segments, stages a compile-spec (resolved time-pairs + stitching config + source uri) to the bucket, dispatches ``worker compile`` on the L4, bucket-polls a done-marker, and returns the same `{status, filepath, download_url}` shape the SPA expects. The worker stitches via the `SHARED _stitch_reel` core (no forked pipeline) and needs no control-plane DB.
- NVENC: `StitchingConfig.encoder + deployment.cloud_stitch_encoder` (default `h264_nvenc`) forwarded to the L4; `VideoCompiler` probes it once and falls back to `libx264` (CPU) everywhere else. Lands the `D16(b)-deferred NVENC-behind-a-config-knob` now that the CUJ supplies the cost data; `libx264` stays the parity-safe default.
- validation delegation: `phase_placement.validation` moves the validator suite off the control-plane via the existing `--skip-validation` lever (#513), guarded so an in-process run always validates.
- cloud-serving preset places compile on the L4.

Docs: `docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md` (+ README pointer).
 Tests: new `test_phase_placement.py`, `test_compile_dispatch.py`; additions to `test_compute_backend.py` (`run_compile`), `test_worker.py` (`cmd_compile`),

test_compiler.py (encoder selection), test_api_cloud_dispatch.py (validation delegation). Full suite green (1875 passed), ruff/mypy clean, coverage 85.7%.

Co-authored-by: Claude Opus 4.8 <noreply@anthropic.com>

```
diff --git a/backend/orchestration.py b/backend/orchestration.py
index 8b7851e..46ffaa4 100644
--- a/backend/orchestration.py
+++ b/backend/orchestration.py
@@ -26,11 +26,20 @@ import os
 import re
 import tempfile
 import threading
+import uuid
 from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
 from urllib.parse import quote

 from backend import storage
-from backend.config import AppConfig, StitchingConfig
+from backend.config import (
+    PHASE_COMPILE,
+    PHASE_SEGMENT,
+    PHASE_VALIDATION,
+    PLACE_CLOUD_L4,
+    AppConfig,
+    StitchingConfig,
+    resolve_phase_placement,
+)
 from backend.pipeline.segmenters.base import segmenter_registry
 from backend.pipeline.stitcher.watermark_i18n import localize_watermark
 from backend.pipeline.validators.base import ValidationResult, registry
@@ -206,6 +215,24 @@ def _cloud_available(cfg) -> bool:
     return bool(has_job and out_root)

+def _will_detect_on_cloud(cfg, req_compute: Optional[str]) -> bool:
+    """Would DETECT for this request run on the L4 worker (vs in-process)? Mirrors the
+    ``_maybe_dispatch_remote`` decision WITHOUT side effects (#521), so ``_execute_processing``
+    can
+    + decide whether to DELEGATE validation to the worker (``phase_placement.validation``). A
+    per-request
+    + picker choice wins; otherwise ``phase_placement.segment`` (whose legacy default is cloud
+    iff
+    + ``compute_target==cloud_run``) gates it, and cloud must actually be available. A pre-
+    dispatch reject
+    + (e.g. cloud requested but unconfigured) counts as NOT-on-cloud ? nothing runs remotely."""
+    choice = (req_compute or "").strip().lower()
+    if choice == "local":
+        return False
+    if choice == "cloud":
+        return _cloud_available(cfg)
+    return (
+        resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
+        and _cloud_available(cfg)
+    )
+
+def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
+    """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
+    minute-granular: the control plane knows ELAPSED time, not frames ? frame-level % comes
@@ -505,11 +532,19 @@ def _maybe_dispatch_remote(
    )
    compute = get_compute_backend(cfg, target_override="cloud_run")
    else:
-        compute = get_compute_backend(
```

```

-         cfg
-     ) # server default (default-OFF unless cloud_run)
+     # No explicit picker ? consult phase_placement.segment (#521). Its legacy default is
+     # cloud_run_l4 IFF compute_target==cloud_run, so this preserves today's behaviour;
setting
+     # phase_placement.segment relocates DETECT by config alone (e.g. force it back on the
+     # control-plane, or onto the L4 on a default-local box wired for cloud).
+     if (
+         resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
+         and _cloud_available(cfg)
+     ):
+         compute = get_compute_backend(cfg, target_override="cloud_run")
+     else:
+         compute = None
    if compute is None:
-         return None # default-OFF ? run the in-process segmenter below (unchanged)
+         return None # default-OFF / placed on control_plane ? the in-process segmenter below

    storage_cfg = getattr(cfg, "storage", None)

@@ -623,12 +658,20 @@ def _maybe_dispatch_remote(
    # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
    # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
    # then worker-rejected at its default 20.0 after a successful GPU dispatch).
+    # Who validates? phase_placement.validation (#521). DEFAULT (control_plane): the control-
plane is
+    # the authoritative gate ? it already validated with its resolved config (only a PASS
reaches
+    # here), so the worker SKIPS its redundant re-validation (#513). If validation is placed on
the
+    # L4 (cloud_run_l4), the control-plane delegated it (it did NOT run the validators ? see
+    # _execute_processing), so the worker must NOT skip: it becomes the gate.
+    worker_skips_validation = (
+        resolve_phase_placement(cfg, PHASE_VALIDATION) != PLACE_CLOUD_L4
+    )
    spec = ComputeJobSpec(
        video_uri=req.video_path,
        out_uri=out_root,
        segmenter=seg_name,
        config_overrides=tuple(overrides),
-        skip_validation=True,
+        skip_validation=worker_skips_validation,
    )
    try:
        # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
@@ -811,12 +854,33 @@ def _execute_processing(
    # ``force=true`` bypasses the reuse and re-writes a fresh verdict below (mirrors how it
bypasses
    # the whole-pipeline cache). Composes with #513 (the worker skips its own re-validation
on a
    # cloud dispatch), so a post-failure reupload reaches segmentation in ~0 validation
time.
-    val_fingerprint = registry.config_fingerprint(config)
+    #
+    # #521 phase_placement.validation: when validation is placed on the L4 AND detection
will
+    # actually dispatch there, the control-plane DELEGATES validation to the worker ? it
does NOT
+    # run the validator suite here (the whole point is to move the CPU-bound validation OFF
the
+    # 2-vCPU control-plane). The worker then validates (skip_validation=False) and rejects
a bad
+    # clip via a non-zero rc, which _maybe_dispatch_remote surfaces as a failed job. When

```



```

detection
+     # is NOT on the cloud (in-process), the control-plane always validates ? there is no
worker to
+     # delegate to (the config guard makes validation=cloud_run_l4 require a cloud segment).
+     validate_on_control_plane = (
+         resolve_phase_placement(config, PHASE_VALIDATION) != PLACE_CLOUD_L4
+         or not _will_detect_on_cloud(config, getattr(req, "compute", None))
+     )
+     if not validate_on_control_plane:
+         logger.info(
+             "[API] validation delegated to the L4 worker "
+             "(deployment.phase_placement.validation=cloud_run_l4) ? the control-plane skips
its "
+             "validator suite; the worker is the gate for %s.",
+             req.video_path,
+         )
+     val_fingerprint = (
+         registry.config_fingerprint(config) if validate_on_control_plane else ""
+     )
+     # The whole read ? fingerprint-match ? replay is guarded: ANY cache fault (unreadable
row,
+     # corrupt or schema-incompatible stored results) degrades to a real validation rather
than
+     # failing the job. The cache is a pure speedup ? never a new failure mode.
+     reused_validation = False
-     if not req.force:
+     if validate_on_control_plane and not req.force:
+         try:
+             cached = db.get_validation_cache(video_id)
+             if cached is not None and cached.get("fingerprint") == val_fingerprint:
@@ -841,11 +905,11 @@ def _execute_processing(
+                 video_id,
+                 exc_info=True,
+             )
-             if not reused_validation:
+             if validate_on_control_plane and not reused_validation:
+                 logger.info(f"Running validations for {req.video_path}...")
+                 val_results, val_context = registry.run_all_with_context(local_video, config)
+                 failures = [r for r in val_results if not r.passed]
-             if not reused_validation:
+             if validate_on_control_plane and not reused_validation:
+                 # Persist the verdict keyed by (content, config) so the NEXT identical reupload
skips the
+                 # decode. Best-effort (the writer swallows + logs its own faults); a miss just re-
validates.
+                 db.set_validation_cache(
@@ -1114,28 +1178,34 @@ def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
+                 return None

-def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
-     """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments, stitch the
reel,
-     and return the {status, filepath, download_url} result the SPA polls for. Returns a
-     ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a stitch
error)
-     so the SPA always gets a clean verdict instead of a worker-crash error."""
-     import backend.main as _main # call-time seam: VideoCompiler stays patchable on
backend.main
+def _compile_wait_message(elapsed_sec: float) -> str:
+     """The live job.progress line while an L4 compile/stitch is in flight (#521; mirrors
+     ``_wait_progress_message``). Minute-granular; under a minute reads as cold-start."""
+     minutes = int(elapsed_sec) // 60
+     if minutes < 1:
+         return "stitching on cloud GPU (NVENC) ? starting up"

```

```

+   return f"stitching on cloud GPU (NVENC) ? {minutes} min elapsed"

-   notify = progress or (lambda stage: None)
-   params = json.loads(job.get("params") or "{}")
-   video_id = job.get("video_id") or ""
-   segment_ids = params.get("segment_ids") or []
-   locale = params.get("locale")

-   notify("resolving")
-   try:
-       video, kept, out_name = _resolve_compile(
-           db, cfg, video_id, segment_ids, params.get("output_filename", "")
-       )
-   except _CompileError as e:
-       return {"status": "failed", "message": e.detail, "video_id": video_id}
+def _stitch_reel(
+    cfg,
+    stitching,
+    video_id: str,
+    source_ref: str,
+    time_pairs: List[Tuple[float, float]],
+    out_name: str,
+    locale: Optional[str],
+    notify,
+) -> Dict[str, Any]:
+    """Stitch + mirror-upload core, shared by the control-plane compile handler AND the L4
worker
+    (#521). ``time_pairs`` are already RESOLVED (the caller filtered via ``_resolve_compile``),
so this
+    needs NO control-plane DB ? the worker reads them from the staged spec. Builds the
VideoCompiler
+    with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the source, runs
the
+    trim/watermark/concat, mirrors the reel to ``<output_root>/highlights/<video_id>/<name>``,
and
+    returns the ``{status, filepath, download_url, video_id, reel_uri}`` the SPA polls for.
Returns a
+    ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg
failure)."""
+    import backend.main as _main # call-time seam: VideoCompiler stays patchable on
backend.main

-   stitching = getattr(cfg, "stitching", None) or StitchingConfig()
-   output_dir = (
-       getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
-   )
@@ -1147,8 +1217,8 @@ def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) ->
Dict[str, A
-       begin_padding=stitching.begin_padding,
-       end_padding=stitching.end_padding,
-       watermark=localized_watermark,
+       encoder=getattr(stitching, "encoder", "libx264"),
+   )
-   time_pairs = [(s["start_time"], s["end_time"]) for s in kept]

-   notify("stitching")
-   local_src = ""
@@ -1157,7 +1227,7 @@ def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) ->
Dict[str, A
-       # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-scratch for
a URI (the
-       # same seam the process path uses); without it, compiling a bucket-ingested video can't
open gs://.
-       local_src = storage.acquire_local_input(
-           video["filepath"], getattr(cfg, "storage", None)

```

```

+         source_ref, getattr(cfg, "storage", None)
+     )
+     # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count (for
remote
+     # dispatch) doesn't run multiple compiles at once on the box (see _LOCAL_COMPUTE_LANE).
@@ -1175,13 +1245,14 @@ def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) ->
Dict[str, A
+     finally:
+         if local_src:
+             storage.cleanup_acquired_local_input(
-                 video["filepath"], local_src, getattr(cfg, "storage", None)
+                 source_ref, local_src, getattr(cfg, "storage", None)
+             )

+     # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed URL ?
Cloud Run
+     # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp copy dies
when the
+     # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch (the
endpoint
-     # still falls back to the local file on the same instance).
+     # still falls back to the local file on the same instance). On the L4 worker the mirror is
how the
+     # reel reaches the bucket at all (the worker's local file is discarded when the Job exits).
+     reel_uri = reel_object_uri(cfg, video_id, out_name)
+     if reel_uri:
+         try:
@@ -1201,4 +1272,189 @@ def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) ->
Dict[str, A
+         "filepath": out_path,
+         "download_url": download_url,
+         "video_id": video_id,
+         "reel_uri": reel_uri or "",
+     }
+
+
+def _dispatch_compile_remote(
+    cfg,
+    db,
+    video_id: str,
+    segment_ids: List[int],
+    output_filename: str,
+    locale: Optional[str],
+    notify,
+) -> Dict[str, Any]:
+    """#521: run the compile/stitch on the L4 worker instead of the CPU-only control-plane. The
+    control-plane owns the DB, so it RESOLVES the reel's segments here (`_resolve_compile`),
+    stages a
+    small compile-spec (resolved time-pairs + the resolved stitching/watermark config + the
+    source URI)
+    to the bucket, dispatches a ``compile`` Cloud Run Job on the L4 (NVENC), bucket-polls the
+    done-marker, and returns the same ``{status, filepath, download_url}`` shape the SPA
+    expects. The
+    worker needs NO control-plane DB ? everything it needs rides the spec. Mirrors
+    ``_maybe_dispatch_remote`` (the DETECT dispatch)."""
+    from backend.compute import CompileJobSpec, get_compute_backend
+
+    notify("resolving")
+    try:
+        video, kept, out_name = _resolve_compile(
+            db, cfg, video_id, segment_ids, output_filename
+        )
+    except _CompileError as e:
+        return {"status": "failed", "message": e.detail, "video_id": video_id}
+

```

```

+     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
+     storage_cfg = getattr(cfg, "storage", None)
+     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
+     compute = get_compute_backend(cfg, target_override="cloud_run") if out_root else None
+     if compute is None:
+         # Compile was placed on the L4 but the cloud dispatch isn't actually wired here (no
output
+         # bucket / Cloud Run env). Fall back to the in-process stitch rather than fail ? a slow
reel
+         # beats no reel. (The placement guard means this is only reachable in a mis-provisioned
setup.)
+         logger.warning(
+             "[compile] compile placed on the L4 but no cloud backend available (out_root=%r) ?
"
+             "stitching in-process on the control-plane instead.",
+             out_root,
+         )
+         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
+         return _stitch_reel(
+             cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale, notify
+         )
+
+     # Build the compile-spec the worker consumes. time-pairs are JSON lists (frozen-safe on the
wire);
+     # the stitching config is the control-plane's RESOLVED one, with the encoder swapped to the
L4's
+     # (deployment.cloud_stitch_encoder, NVENC by default) so the worker hardware-encodes the
reel.
+     stitch_dump = stitching.model_dump()
+     stitch_dump["encoder"] = getattr(
+         getattr(cfg, "deployment", None), "cloud_stitch_encoder", "h264_nvenc"
+     )
+     spec_payload = {
+         "video_id": video_id,
+         "video_uri": video["filepath"],
+         "output_name": out_name,
+         "locale": locale,
+         "time_pairs": [
+             [float(s["start_time"]), float(s["end_time"])] for s in kept
+         ],
+         "stitching": stitch_dump,
+     }
+     token = uuid.uuid4().hex
+     spec_uri = f"{out_root.rstrip('/')}/_compile/{token}/spec.json"
+     try:
+         notify("staging compile spec")
+         with tempfile.TemporaryDirectory(prefix="cr-compile-") as td:
+             local_spec = os.path.join(td, "spec.json")
+             with open(local_spec, "w", encoding="utf-8") as f:
+                 json.dump(spec_payload, f)
+             storage.put(local_spec, spec_uri, config=_storage_settings(cfg))
+     except Exception as e: # noqa: BLE001 ? a staging failure is a clean job failure, not a
crash
+         logger.error(
+             "[compile] failed to stage the compile spec to %s: %s", spec_uri, e, exc_info=True
+         )
+         return {
+             "status": "failed",
+             "message": f"cloud compile: could not stage the compile spec: {e}",
+             "video_id": video_id,
+         }
+
+     # Forward the bucket coordinates so the worker (no DEPLOYMENT_PROFILE) can read the spec,
mirror the
+     # reel, and write the marker ? mirrors how _maybe_dispatch_remote forwards the WASB knobs

```

```

for detect.
+ overrides = [f"storage.output_root={out_root}"]
+ backend_name = (getattr(storage_cfg, "backend", "") or "").strip()
+ if backend_name:
+     overrides.append(f"storage.backend={backend_name}")
+ spec = CompileJobSpec(
+     out_uri=out_root,
+     spec_uri=spec_uri,
+     video_id=video_id,
+     output_name=out_name,
+     config_overrides=tuple(overrides),
+ )
+
+ notify("dispatching stitch (cloud_run)")
+ logger.info(
+     "[compile] dispatching stitch for video_id=%s -> Cloud Run Job (out=%s, %d clip(s))",
+     video_id,
+     out_root,
+     len(kept),
+ )
+ try:
+     result = compute.run_compile(
+         spec, on_wait=lambda elapsed: notify(_compile_wait_message(elapsed))
+     )
+ except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job failure,
not a 500
+     logger.error("[compile] cloud compile dispatch error: %s", e, exc_info=True)
+     return {
+         "status": "failed",
+         "message": f"cloud compile dispatch error: {e}",
+         "video_id": video_id,
+     }
+ if result.returncode != 0:
+     msg = str(
+         result.report.get("message")
+         or f"cloud compile job failed (returncode {result.returncode})."
+     )
+     return {"status": "failed", "message": msg, "video_id": video_id}
+
+ download_url = str(
+     result.report.get("download_url")
+     or f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
+ )
+ logger.info(
+     "[COMPILE] cloud_run stitched: execution=%s out=%s video_id=%s",
+     result.execution,
+     result.outputs_uri,
+     video_id,
+ )
+ notify("finished")
+ return {
+     "status": "success",
+     "filepath": str(result.report.get("reel_uri") or result.outputs_uri or ""),
+     "download_url": download_url,
+     "video_id": video_id,
+     "compute": {
+         "path": "cloud_run",
+         "execution": result.execution,
+         "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
+         "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
+     },
+ }
+
+def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:

```

```

+     """Handler for a ``kind='compile'`` job (#329): re-resolve the segments, stitch the reel,
and return
+     the ``{status, filepath, download_url}`` result the SPA polls for. Returns a
``status='failed'``
+     result for an EXPECTED problem (missing video / no segments / a stitch error) so the SPA
always gets
+     a clean verdict instead of a worker-crash error.
+
+     #521 phase_placement.compile: when compile is placed on the L4 (``cloud_run_l4``) and cloud
+     dispatch is available, the CPU-bound 4K stitch is OFFLOADED to the GPU/NVENC worker
+     (``_dispatch_compile_remote``) ? the 2026-07-08 CUJ measured ~13 min for an 11-clip reel on
the
+     2-vCPU control-plane. DEFAULT (compile on ``control_plane``) stitches in-process here,
unchanged."""
+     notify = progress or (lambda stage: None)
+     params = json.loads(job.get("params") or "{}")
+     video_id = job.get("video_id") or ""
+     segment_ids = params.get("segment_ids") or []
+     locale = params.get("locale")
+     output_filename = params.get("output_filename", "")
+
+     if (
+         resolve_phase_placement(cfg, PHASE_COMPILE) == PLACE_CLOUD_L4
+         and _cloud_available(cfg)
+     ):
+         return _dispatch_compile_remote(
+             cfg, db, video_id, segment_ids, output_filename, locale, notify
+         )
+
+     notify("resolving")
+     try:
+         video, kept, out_name = _resolve_compile(
+             db, cfg, video_id, segment_ids, output_filename
+         )
+     except _CompileError as e:
+         return {"status": "failed", "message": e.detail, "video_id": video_id}
+
+     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
+     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
+     return _stitch_reel(
+         cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale, notify
+     )

```

14. assistant (2026-07-08T06:04:49.564Z)

Now the config diffs.

15. user (2026-07-08T06:04:51.211Z)

```

commit dac6736cb23a681135b707d768014820b239a981
Author: Avi <avi.dullu@gmail.com>
Date:   Wed Jul 8 11:29:33 2026 +0530

```

perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement (#521) (#525)

The compile/stitch phase ran in-process on the CPU-only 2-vCPU control-plane; the 2026-07-08 live CUJ (rev rally-control-plane-00019, S9 of the validation-latency runlog) measured ~12m55s for an 11-clip / ~80s reel from a 1.83 GiB 4K source (~45-90s per ~7s clip ? the per-clip branding watermark forces a full 4K re-encode with no NVENC). Validation (183s) also runs there; both serialize on _LOCAL_COMPUTE_LANE, so one run pins the control-plane ~16 min ? won't scale past a handful of concurrent users.

- Config-driven placement: deployment.phase_placement {validation|segment|compile}

- ? control_plane | cloud_run_l4, resolved by backend/config/placement.py. Unset ? today's placement, byte-identical. A config-load guard rejects the "nobody validates" footgun (validation on L4 without a cloud segment).
- compile ? L4: a kind='compile' Cloud Run dispatch mirroring _maybe_dispatch_remote. The control-plane resolves the reel's segments, stages a compile-spec (resolved time-pairs + stitching config + source uri) to the bucket, dispatches `worker compile` on the L4, bucket-polls a done-marker, and returns the same {status, filepath, download_url} shape the SPA expects. The worker stitches via the SHARED _stitch_reel core (no forked pipeline) and needs no control-plane DB.
- NVENC: StitchingConfig.encoder + deployment.cloud_stitch_encoder (default h264_nvenc) forwarded to the L4; VideoCompiler probes it once and falls back to libx264 (CPU) everywhere else. Lands the D16(b)-deferred NVENC-behind-a-config-knob now that the CUJ supplies the cost data; libx264 stays the parity-safe default.
- validation delegation: phase_placement.validation moves the validator suite off the control-plane via the existing --skip-validation lever (#513), guarded so an in-process run always validates.
- cloud-serving preset places compile on the L4.

Docs: docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md (+ README pointer).

Tests: new test_phase_placement.py, test_compile_dispatch.py; additions to test_compute_backend.py (run_compile), test_worker.py (cmd_compile), test_compiler.py (encoder selection), test_api_cloud_dispatch.py (validation delegation). Full suite green (1875 passed), ruff/mypy clean, coverage 85.7%.

Co-authored-by: Claude Opus 4.8 <noreply@anthropic.com>

```
diff --git a/backend/config/__init__.py b/backend/config/__init__.py
index 3f722c9..14cf483 100644
--- a/backend/config/__init__.py
+++ b/backend/config/__init__.py
@@ -19,10 +19,19 @@ from .models import ( # re-export common ones
    Config,
    DeploymentConfig,
    IndexingConfig,
+   PhasePlacementConfig,
    StitchingConfig,
    StorageConfig,
    ValidationConfig,
)
+from .placement import (
+   PHASE_COMPILE,
+   PHASE_SEGMENT,
+   PHASE_VALIDATION,
+   PLACE_CLOUD_L4,
+   PLACE_CONTROL_PLANE,
+   resolve_phase_placement,
+)
from .presets import PROFILE_PRESETS, probe_cuda_available, suggest_profile
from .startup import (
    StartupCheck,
@@ -52,9 +61,16 @@ __all__ = [
    "Config",
    "DeploymentConfig",
    "IndexingConfig",
+   "PhasePlacementConfig",
    "StitchingConfig",
    "StorageConfig",
    "ValidationConfig",
+   "resolve_phase_placement",
+   "PHASE_VALIDATION",
+   "PHASE_SEGMENT",
+   "PHASE_COMPILE",
+   "PLACE_CONTROL_PLANE",
+   "PLACE_CLOUD_L4",
    "PROFILE_PRESETS",
```

```

    "suggest_profile",
    "probe_cuda_available",
diff --git a/backend/config/models.py b/backend/config/models.py
index a7c997c..ce8e98a 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -364,6 +364,14 @@ class StitchingConfig(BaseModel):
    # background-court fragments), so this keeps the reel to the eye-catching long rallies.
    # OFF by default ? the detector output is unchanged, only which rallies reach the reel.
    min_rally_duration: float = Field(default=0.0, ge=0.0)
+   # Video encoder for the per-clip trim re-encode (the branding watermark forces a re-encode;
the
+   # concat stays -c copy). Default ``libx264`` (CPU, works everywhere incl. CI).
``h264_nvenc`` /
+   # ``hevc_nvenc`` are the L4 HARDWARE encoders ? dramatically faster on the 4K re-encode
that
+   # dominated the 2026-07-08 CUJ stitch (#521). The compiler probes the requested encoder
ONCE and
+   # falls back to libx264 if this ffmpeg build can't run it, so a non-GPU box never breaks.
The
+   # cloud compile-dispatch forwards ``deployment.cloud_stitch_encoder`` onto this for the L4
run;
+   # the control-plane's own local stitch keeps libx264.
+   encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "libx264"
+   watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)

@@ -451,6 +459,21 @@ class StorageConfig(BaseModel):
    gdrive_api: dict[str, Any] = Field(default_factory=dict)

+class PhasePlacementConfig(BaseModel):
+   """Per-phase machine placement (#521): move validation / segment(detect) / compile(stitch)
+   between the control-plane and the L4 Cloud Run worker **by config, no code change**.
+
+   Each field is ``None`` (? the legacy default for that phase ? see
+   ``backend/config/placement.py``) or an explicit ``"control_plane"`` / ``"cloud_run_l4"``.
+   ``control_plane`` runs the phase in-process on the CPU-only control-plane; ``cloud_run_l4``
+   dispatches it to the GPU/NVENC worker Job. Unset ? byte-identical to pre-#521 behaviour.
+   Resolve via ``backend.config.resolve_phase_placement(cfg, phase)`` ? never read these
+   directly."""
+
+   validation: Optional[Literal["control_plane", "cloud_run_l4"]] = None
+   segment: Optional[Literal["control_plane", "cloud_run_l4"]] = None
+   compile: Optional[Literal["control_plane", "cloud_run_l4"]] = None
+
+   class DeploymentConfig(BaseModel):
+       """Deployment profile ? one switch that selects a coherent bundle of compute + storage
+       knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
@@ -481,6 +504,35 @@ class DeploymentConfig(BaseModel):
    # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect + queue/cold-
start,
    # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid GPU run.
    remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
+   # Per-phase machine placement (#521). Unset ? today's placement (validation + compile on
the
+   # control-plane, segment on the L4 iff compute_target==cloud_run). Set a phase to relocate
it by
+   # config alone. Resolve via ``backend.config.resolve_phase_placement`` (never read the sub-
fields
+   # directly ? the resolver folds in the legacy compute_target default).
+   phase_placement: PhasePlacementConfig = Field(default_factory=PhasePlacementConfig)
+   # Video encoder the cloud compile-dispatch forwards onto ``stitching.encoder`` for the L4
stitch

```



```

+ # (#521). ``h264_nvenc`` uses the L4's hardware encoder for the 4K re-encode the watermark
forces;
+ # set ``libx264`` to run the L4 stitch on CPU. Only consulted when compile is placed on the
L4;
+ # the control-plane's own local stitch always uses ``stitching.encoder`` (libx264 by
default).
+ cloud_stitch_encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "h264_nvenc"
+
+ @model_validator(mode="after")
+ def _validation_placement_needs_cloud_segment(self) -> "DeploymentConfig":
+     # A clip can only be validated ON the worker if the worker also DETECTS it there ? the
worker
+     # is only launched for the segment(detect) phase. So validation=cloud_run_l4 with
segment on
+     # the control-plane would leave NOBODY validating (the control-plane skips, and no
worker runs)
+     # ? reject that footgun at config load rather than ship a silently-unvalidated clip.
+     if self.phase_placement.validation == "cloud_run_l4":
+         seg = self.phase_placement.segment or (
+             "cloud_run_l4" if self.compute_target == "cloud_run" else "control_plane"
+         )
+         if seg != "cloud_run_l4":
+             raise ValueError(
+                 "deployment.phase_placement.validation='cloud_run_l4' requires segment to
also "
+                 "run on the L4 ? the worker can only validate a clip it also detects. Set "
+                 "phase_placement.segment='cloud_run_l4' (or compute_target='cloud_run'), or
keep "
+                 "validation on the control_plane."
+             )
+     return self

```

```

class BillingConfig(BaseModel):
diff --git a/backend/config/placement.py b/backend/config/placement.py
new file mode 100644
index 0000000..cda3ca5
--- /dev/null
+++ b/backend/config/placement.py
@@ -0,0 +1,59 @@
+"""Phase ? machine placement resolution (#521).
+
+
+A *phase placement* answers one question per pipeline phase ? **where does the heavy compute
for
+this phase run? ? with a config knob (``deployment.phase_placement``) instead of code. The
three
+placeable phases and their values:
+
+
+ validation ? control_plane | cloud_run_l4
+ segment ? control_plane | cloud_run_l4 (a.k.a. "detect")
+ compile ? control_plane | cloud_run_l4 (a.k.a. "stitch")
+
+
+ ``control_plane`` runs the phase IN-PROCESS on the (CPU-only) control-plane service;
+ ``cloud_run_l4``
+ dispatches it to the GPU/NVENC Cloud Run worker Job (the same L4 detection already uses). This
module
+ is the SINGLE source of truth so ``backend.orchestration`` (segment + compile) and the worker
never
+ re-derive the mapping.
+
+
+ **Default = today's behaviour (pure / additive).** When a phase's knob is unset (``None``) the
+ resolver reproduces the pre-#521 placement:
+
+
+ validation ? control_plane (the authoritative gate,
+ #512/#513)

```

```

+     segment      ? cloud_run_l4 IFF deployment.compute_target == "cloud_run", else
control_plane
+     compile      ? control_plane                                     (the in-process ffmpeg
stitch)
+
+so a config with no ``phase_placement`` block behaves byte-identically to before this module
existed.
+"""
+
+from __future__ import annotations
+
+from typing import Any
+
+# Phase identities (kept as constants so callers don't stringly-type the phase names).
+PHASE_VALIDATION = "validation"
+PHASE_SEGMENT = "segment"
+PHASE_COMPILE = "compile"
+PHASES: tuple[str, ...] = (PHASE_VALIDATION, PHASE_SEGMENT, PHASE_COMPILE)
+
+# Placement targets.
+PLACE_CONTROL_PLANE = "control_plane"
+PLACE_CLOUD_L4 = "cloud_run_l4"
+PLACEMENTS: tuple[str, ...] = (PLACE_CONTROL_PLANE, PLACE_CLOUD_L4)
+
+def resolve_phase_placement(cfg: Any, phase: str) -> str:
+    """Where does ``phase`` run ? ``"control_plane"`` or ``"cloud_run_l4"``?
+
+    Precedence: an explicit ``deployment.phase_placement.<phase>`` wins; otherwise the legacy
default
+    (mirrors the pre-#521 behaviour, see the module docstring). Never raises ? an absent
+    ``deployment`` / ``phase_placement`` degrades to the legacy default.
+    """
+    deployment = getattr(cfg, "deployment", None)
+    placement = getattr(deployment, "phase_placement", None)
+    explicit = getattr(placement, phase, None) if placement is not None else None
+    if explicit:
+        return str(explicit)
+    # Legacy defaults ? reproduce today's placement exactly.
+    if phase == PHASE_SEGMENT:
+        compute_target = (getattr(deployment, "compute_target", "") or "").strip()
+        return PLACE_CLOUD_L4 if compute_target == "cloud_run" else PLACE_CONTROL_PLANE
+    # validation + compile default to the control-plane (their pre-#521 home).
+    return PLACE_CONTROL_PLANE
diff --git a/backend/config/presets.py b/backend/config/presets.py
index 77b20e8..0ea0c88 100644
--- a/backend/config/presets.py
+++ b/backend/config/presets.py
@@ -49,7 +49,15 @@ PROFILE_PRESETS: dict[str, dict[str, Any]] = {
     "storage": {"backend": "local"},
 },
 "cloud-serving": {
-    "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
+    "deployment": {
+        "profile": "cloud-serving",
+        "compute_target": "cloud_run",
+        # #521: run the compile/stitch on the L4 worker (NVENC + 8 vCPU), not the CPU-only
+        # 2-vCPU control-plane where the 2026-07-08 CUJ measured ~13 min for an 11-clip 4K
reel.
+        # (validation stays on the control-plane ? the authoritative #512/#513 gate;
segment is
+        # already cloud_run_l4 via compute_target.)
+        "phase_placement": {"compile": "cloud_run_l4"},
+    },
+    # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor (D11)

```

```
# wasb GPU-feed tuning: the parity defaults (batch 8, no prefetch) left the paid L4
# <1% utilized on the first real serving run (2026-07-03) ? the pipeline was
```

16. assistant (2026-07-08T06:05:13.431Z)

Let me read the full current source of the key files to trace token/marker semantics and the concat step.

17. user (2026-07-08T06:05:14.334Z)

```
1      """`CloudRunCompute` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2      for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4      Flow (the cloud-serving compute path):
5      1. **Dispatch** a Cloud Run Job execution whose container runs `python -m
backend.worker infer
6          --video-uri <gs> --out-uri <gs> --done-uri <gs> ?` (the SAME worker, on the L4
GPU).
7      2. **Bucket-poll** `done_uri` via `execution_policy.poll_until` + the storage
layer (D16 ? the
8          completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9          it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10         cold-start is absorbed by the async poll. On a poll TIMEOUT (`max_wait_sec` can
undercut the
11         job's own `--task-timeout` budget) the shim does ONE final marker check ? a slow-
but-healthy
12         worker that finished just past the deadline is rescued, not wasted ? then best-
effort CANCELS
13         the still-running execution so the L4 doesn't keep billing until `--task-
timeout`.
14         *Fast negative path:* while the marker stays the SOLE success signal, each poll
also probes the
15         Cloud Run Executions API ? if the execution has TERMINATED without ever writing a
marker (a
16         worker crash/OOM/argparse-abort before the marker push), the shim fails fast +
diagnostically
17         (`RemoteExecutionFailed`) instead of spinning the full ~65-min budget on a futile
wait.
18     3. **Read** the marker (it carries `video_id` + the worker's returncode) ? fetch
19         `outputs/<video_id>/report.json` ? `ComputeResult`.
20
21     The dispatched container is forced to run **INLINE** (`compute_target=local_gpu` +
native detector)
22     so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
23
24     The Cloud Run trigger is an **injected client** (`CloudRunJobClient`) so the whole
shim is unit-tested
25     with a fake (no GCP). The real `GoogleCloudRunJobClient` lazy-imports `google-cloud-
run` and is
26     owner-verified on the M7 real run.
27     """
28
29     from __future__ import annotations
30
31     import json
32     import logging
33     import os
34     import tempfile
35     import uuid
36     from abc import ABC, abstractmethod
37     from typing import Any, Callable, List, Optional
```

```

38
39     from backend.compute.base import (
40         ComputeBackend,
41         ComputeJobSpec,
42         ComputeResult,
43         RemoteExecutionFailed,
44     )
45     from backend.infrastructure.execution_policy import (
46         DEFAULT_POLICY,
47         ExecutionPolicy,
48         PolicyTimeout,
49         poll_until,
50     )
51     from backend.storage import fetch, get_storage, parse_uri
52
53     LOG = logging.getLogger("compute.cloud_run")
54
55     # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
56     # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
57     # runner.)
58     _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
59         "deployment.compute_target=local_gpu",
60         "indexing.detector_impl=native",
61     )
62
63     class CloudRunJobClient(ABC):
64         """Triggers a Cloud Run **Job** execution with per-execution container arg
65         overrides.
66
67         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
68         implementation
69         overrides the job's container ``args`` for this execution and starts it."""
70
71         @abstractmethod
72         def run_job(
73             self,
74             job_name: str,
75             argv: List[str],
76             *,
77             region: str,
78             project: Optional[str] = None,
79         ) -> str:
80             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
81             return an
82             execution id/name (for logs). Does NOT block on completion (the caller bucket-
83             polls)."""
84
85         @abstractmethod
86         def cancel_execution(self, execution: str) -> None:
87             """Cancel a running execution by the name ``run_job`` returned. Called when the
88             bucket-poll
89             times out so the abandoned execution doesn't keep billing the GPU until
90             ``--task-timeout``.
91             May raise ? the caller treats every failure as best-effort (the original poll
92             timeout is
93             NEVER masked by a cancel error)."""
94
95         def execution_terminal_state(self, execution: str) -> Optional[str]:
96             """Return the execution's TERMINAL state as a short label (``"Failed"`` /
97             ``"Cancelled"`` /
98             ``"Succeeded"``) if it has finished, else ``None`` (still pending/running, or
99             the state
100             can't be determined).

```

```

92
93     ADVISORY, not authoritative: it feeds ``run_infer``'s fast negative path so a
worker that
94     terminated WITHOUT writing the done-marker (crash/OOM/pre-marker abort) fails
fast instead
95     of bucket-polling the full budget. The done-marker stays the ONLY success
signal.
96
97     NON-abstract with a default of ``None`` (additive / default-OFF): a client that
can't ? or
98     chooses not to ? poll the Executions API degrades transparently to today's
marker-only
99     polling. Implementations MAY raise on a transient API error ? ``run_infer``
treats a raise
100    as ``None`` (unknown ? keep waiting), so a flaky status API never fails a
healthy run. """
101    return None
102
103
104    class GoogleCloudRunJobClient(CloudRunJobClient):
105        """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
106        a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency). """
107
108        def run_job(
109            self,
110            job_name: str,
111            argv: List[str],
112            *,
113            region: str,
114            project: Optional[str] = None,
115        ) -> str:
116            # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
117            # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable
118            # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
119            if not project:
120                raise RuntimeError(
121                    "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "
122                    "to resolve the job path; set it on the control plane (see
get_compute_backend)."
123                )
124            try:
125                from google.cloud import run_v2 # type: ignore[attr-defined]
126            except Exception as exc: # pragma: no cover - real SDK not present in CI
127                raise RuntimeError(
128                    "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
129                    "control plane (it is intentionally NOT a CI/test dependency). "
130                ) from exc
131            client = (
132                run_v2.JobsClient()
133            ) # pragma: no cover - exercised only on the real M7 run
134            name = client.job_path(project, region, job_name)
135            overrides = run_v2.RunJobRequest.Overrides(
136                container_overrides=[
137                    run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
138                ]
139            )
140            op = client.run_job(
141                request=run_v2.RunJobRequest(name=name, overrides=overrides)

```

```

142         )
143         return (
144             getattr(op, "metadata", None)
145             and getattr(op.metadata, "name", "")
146             or "execution"
147         )
148
149     def cancel_execution(self, execution: str) -> None:
150         # run_job falls back to the literal string "execution" when the operation
151         # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK
152         # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI
153         # google-cloud-run is absent).
154         if "/executions/" not in (execution or ""):
155             raise RuntimeError(
156                 f"not a fully-qualified Cloud Run execution name: {execution!r} ?
157                 "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not
158                 "one from the operation metadata, so there is nothing to cancel by
159             )
160         try:
161             from google.cloud import run_v2 # type: ignore[attr-defined]
162         except Exception as exc: # pragma: no cover - real SDK not present in CI
163             raise RuntimeError(
164                 "google-cloud-run is required to cancel a Cloud Run execution; install
165                 "control plane (it is intentionally NOT a CI/test dependency)."
166             ) from exc
167         client = (
168             run_v2.ExecutionsClient()
169         ) # pragma: no cover - exercised only on a real run
170         client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))
171
172     def execution_terminal_state(self, execution: str) -> Optional[str]:
173         # Advisory status probe (see the ABC docstring): fetch the execution and
174         # pure, unit-tested ``_classify_execution_state``. A non-resolvable name /
175         # absent SDK returns
176         # None (can't probe ? keep marker-polling) ? unlike run_job/cancel this never
177         # input, because the probe is best-effort and must not fail a healthy run.
178         if "/executions/" not in (execution or ""):
179             return None
180         try:
181             from google.cloud import run_v2 # type: ignore[attr-defined]
182         except Exception: # pragma: no cover - real SDK not present in CI
183             return None
184         client = (
185             run_v2.ExecutionsClient()
186         ) # pragma: no cover - exercised only on a real run
187         exe = client.get_execution( # pragma: no cover - real run only
188             request=run_v2.GetExecutionRequest(name=execution)
189         )
190         return _classify_execution_state(exe) # pragma: no cover - real run only
191
192     def _classify_execution_state(exe: Any) -> Optional[str]:
193         """Map a Cloud Run ``run_v2.Execution`` to a terminal state label (``"Failed"`` /
194         ``"Cancelled"``
195         / ``"Succeeded"``), or ``None`` if it is still pending/running. PURE (no
196         SDK/network) so it is

```

```

195         unit-tested against both real proto-plus ``Execution`` objects and lightweight fakes
? the actual
196         ``get_execution`` fetch is the only part that can't run in CI.
197
198         ``google-cloud-run`` is a **proto-plus** library, so a ``google.protobuf.Timestamp``
field is
199         marshalled to a native Python ``datetime`` when set and to ``None`` when unset ? NOT
to a proto
200         ``Timestamp`` with a ``.seconds`` field (a ``datetime`` has ``.second``, singular).
Terminality is
201         therefore "``completion_time`` is not None"; the per-task counts (plain ints) then
classify it,
202         with a cancel taking precedence (a cancelled execution reports ``cancelled_count >
0``)."""
203         if getattr(exe, "completion_time", None) is None:
204             return None # no terminal completion timestamp yet ? still pending/running
205         if getattr(exe, "cancelled_count", 0):
206             return "Cancelled"
207         if getattr(exe, "failed_count", 0):
208             return "Failed"
209         if getattr(exe, "succeeded_count", 0):
210             return "Succeeded"
211         return "Failed" # terminal but no positive success ? treat as failed (conservative)
212
213
214     class CloudRunCompute(ComputeBackend):
215         """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
216
217         def __init__(
218             self,
219             *,
220             run_client: CloudRunJobClient,
221             job_name: str,
222             region: str,
223             project: Optional[str] = None,
224             storage_config: Optional[dict] = None,
225             policy: ExecutionPolicy = DEFAULT_POLICY,
226             token: Optional[str] = None,
227             container_overrides: Optional[tuple[str, ...]] = None,
228         ) -> None:
229             self._client = run_client
230             self._job_name = job_name
231             self._region = region
232             self._project = project
233             self._storage_config = storage_config or {}
234             self._policy = policy
235             # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
236             # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
237             self._token = token
238             self._container_overrides = (
239                 _DEFAULT_CONTAINER_OVERRIDES
240                 if container_overrides is None
241                 else tuple(container_overrides)
242             )
243
244         def run_infer(
245             self,
246             spec: ComputeJobSpec,
247             *,
248             on_wait: "Callable[[float], None] | None" = None,
249         ) -> ComputeResult:
250             out_root = spec.out_uri.rstrip("/")
251             token = self._token or uuid.uuid4().hex

```

```

252     done_uri = f"{out_root}/_dispatch/{token}.done"
253     argv: List[str] = [
254         "infer",
255         "--video-uri",
256         spec.video_uri,
257         "--out-uri",
258         spec.out_uri,
259         "--done-uri",
260         done_uri,
261     ]
262     if spec.segmenter:
263         argv += ["--segmenter", spec.segmenter]
264     if spec.skip_validation:
265         # The control-plane already validated + only dispatches on PASS ? tell the
worker not to
266         # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
267         argv.append("--skip-validation")
268     for kv in (*spec.config_overrides, *self._container_overrides):
269         argv += ["--set", kv]
270
271     execution = self._client.run_job(
272         self._job_name, argv, region=self._region, project=self._project
273     )
274     LOG.info(
275         "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
276         self._job_name,
277         execution,
278         done_uri,
279     )
280
281     try:
282         marker = poll_until(
283             lambda: self._poll_check(done_uri, str(execution or "")),
284             self._policy,
285             label="cloud-run-infer",
286             logger=LOG,
287             # Live heartbeat (#482): each still-waiting tick reaches the caller so
288             # job.progress moves during the whole GPU run instead of freezing.
289             on_tick=on_wait,
290         )
291     except PolicyTimeout as timeout_exc:
292         # RemoteExecutionFailed (the fast negative path in _poll_check) is NOT a
PolicyTimeout,
293         # so it propagates past here uncaught ? the dispatch handler maps it to a
distinct,
294         # clearly-labelled failure instead of the ~65-min opaque timeout this branch
handles.
295         marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
296     video_id = str(marker.get("video_id", ""))
297     # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
298     # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
299     rc = int(marker.get("returncode", 1))
300     if not video_id:
301         LOG.warning(
302             "cloud-run: job=%s completion marker present but empty/unreadable (no "
303             "video_id) ? treating as failure",
304             self._job_name,
305         )
306         rc = rc or 1
307     outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
308     report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
309     LOG.info(

```



```

310         "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
311     )
312     return ComputeResult(
313         returncode=rc,
314         outputs_uri=outputs_uri,
315         video_id=video_id,
316         report=report,
317         execution=str(execution or ""),
318     )
319
320     def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
321         """One bucket-poll iteration, hardened with an Executions-API fast negative
322         path.
323         The done-marker is the SOLE success signal and is checked FIRST ? a present
324         marker always wins (even if the execution's status later flips), so a worker that wrote a
325         SUCCESS *or a FAILURE* marker resolves through the normal path with its real returncode. Only
326         when the marker is absent do we consult the execution status: a worker that TERMINATED
327         without ever writing a marker (crash / OOM / an old image argparse-aborting on an unknown
328         flag) would otherwise make this poll spin the full ``remote_poll_max_wait_sec`` (~65 min)
329         before failing opaquely. Detecting that terminal state collapses the whole class to a prompt,
330         actionable failure.
331
332         Returns the marker dict (resolve the poll), ``None`` (keep waiting), or raises
333         ``RemoteExecutionFailed`` (abort the poll immediately ? the execution is over,
334         no marker will ever appear)."""
335         marker = self._read_marker(done_uri)
336         if marker is not None:
337             return marker # success signal ? authoritative even if status later flips
338             state = self._terminal_execution_state(execution)
339             if state is None:
340                 return None # still pending/running, or status unknown ? keep bucket-
341                 polling
342                 # The execution is terminal but no marker is present. Re-check the marker ONCE
343                 to close the # race where the worker wrote it between our two reads (mirrors the post-timeout
344                 rescue in # _handle_poll_timeout). If it is genuinely absent, the process is gone and no
345                 marker will # EVER appear ? fail fast + diagnostically rather than wait out the whole poll
346                 budget.
347                 marker = self._read_marker(done_uri)
348                 if marker is not None:
349                     return marker
350                 raise RemoteExecutionFailed(
351                     f"cloud-run: job={self._job_name} execution={execution or '<unknown>'}
352                     reached " f"terminal state {state!r} with NO completion marker ? the worker exited
353                     before " "writing one (crash / OOM / pre-marker abort, or a failed marker upload on
354                     an " "otherwise-Succeeded run). The execution is over, so the marker will never
355                     appear; " "inspect the Cloud Run Job execution log to diagnose: "
356                     f"``gcloud run jobs executions describe {execution or '<execution>'}`` "
357                     f"--region {self._region}``."
358                 )

```

```

357
358     def _terminal_execution_state(self, execution: str) -> Optional[str]:
359         """Best-effort wrapper over the client's ``execution_terminal_state`` probe. A
360         raising or flaky status API must NEVER kill a healthy wait (the done-marker stays the sole
361         authority), so any exception ? and an empty/unresolvable execution name ? degrades to
362         ``None`` (unknown ? keep polling) and is DEBUG-logged, exactly today's marker-only behaviour."""
363         if not execution:
364             return None
365         try:
366             return self._client.execution_terminal_state(execution)
367         except Exception: # noqa: BLE001 ? advisory probe: never let it kill the
368             LOG.debug(
369                 "cloud-run: execution status probe failed for %s ? treating as unknown",
370                 execution,
371                 exc_info=True,
372             )
373             return None
374
375     def _handle_poll_timeout(
376         self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
377     ) -> dict:
378         """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.
379
380         The poll budget (``max_wait_sec``, default 1800s) can undercut the deployed
381         job's own ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past
382         the deadline: ONE final marker check rescues that completed GPU run instead of wasting it. If
383         the marker is genuinely absent, the execution is still running ? best-effort cancel it
384         (otherwise the L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
385         carrying the execution name so the operator can verify the state in the GCP console. A cancel
386         failure NEVER masks the original timeout."""
387         try:
388             late = self._read_marker(done_uri)
389         except Exception: # noqa: BLE001 ? the final check is best-effort too: absence
390             ? failure
391             LOG.warning(
392                 "cloud-run: final post-timeout marker check failed ? treating as
393                 absent",
394                 exc_info=True,
395             )
396             late = None
397         if late is not None:
398             LOG.warning(
399                 "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
400                 check "
401                 "? rescuing the completed result",
402                 self._job_name,
403                 execution,
404             )
405             return late
406         try:
407             self._client.cancel_execution(execution)
408         except Exception as cancel_exc: # noqa: BLE001 ? best-effort: never mask the
409             timeout
410             cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
411             LOG.warning(

```

```

408         "cloud-run: best-effort cancel of execution=%s failed: %s",
409         execution,
410         cancel_exc,
411         exc_info=True,
412     )
413     else:
414         cancel_note = "cancel requested"
415         LOG.info("cloud-run: requested cancel of execution=%s", execution)
416         raise PolicyTimeout(
417             f"{timeout_exc} [cloud-run job={self._job_name}] "
418             f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution"
419             "state in the GCP console]"
420         ) from timeout_exc
421
422     # --- storage-backed completion signal (bucket-poll) ---
423     def _read_marker(self, done_uri: str) -> Optional[dict]:
424         """The done-marker dict if the worker has written it, else None ? poll_until
425         keeps waiting."""
426         ref = parse_uri(done_uri)
427         if not get_storage(ref.backend, config=self._storage_config).exists(ref):
428             return None
429         return (
430             self._read_json(done_uri) or {}
431         ) # empty dict still terminates the poll (present == done)
432
433     def _read_json(self, uri: str) -> dict:
434         # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
435         # ? a None dst
436         # raises ValueError ? unlike local's identity passthrough. Without an explicit
437         # dst this read
438         # would silently return {} for every gs://|s3:// object (the only backends this
439         # shim targets).
440         try:
441             ref = parse_uri(uri)
442             with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
443                 dst = os.path.join(td, ref.name or "obj.json")
444                 local = fetch(uri, dst, config=self._storage_config)
445                 with open(local, "r", encoding="utf-8") as f:
446                     data: Any = json.load(f)
447                 return data if isinstance(data, dict) else {}
448         except Exception:
449             LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
450             return {}

```

18. assistant (2026-07-08T06:05:33.433Z)

Let me read the full current `orchestration.py` around `\_stitch\_reel`, `reel\_object\_uri`, and `\_LOCAL\_COMPUTE\_LANE`, plus the compiler concat step.

19. user (2026-07-08T06:05:34.922Z)

```

1120     the existence check and the signing (extracted here to keep backend/main.py lean ?
1121     P23/P24)."""
1122     reel_uri = reel_object_uri(cfg, video_id, name)
1123     if not reel_uri:
1124         return None
1125     settings = _storage_settings(cfg)
1126     if storage.exists(reel_uri, config=settings):
1127         return storage.signed_url(reel_uri, config=settings, method="GET")
1128     return None
1129
1130     def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:

```

```

1131     """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
1132     and return the {status, filepath, download_url} result the SPA polls for. Returns a
1133     ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
1134     so the SPA always gets a clean verdict instead of a worker-crash error."""
1135     import backend.main as _main # call-time seam: VideoCompiler stays patchable on
backend.main
1136
1137     notify = progress or (lambda stage: None)
1138     params = json.loads(job.get("params") or "{}")
1139     video_id = job.get("video_id") or ""
1140     segment_ids = params.get("segment_ids") or []
1141     locale = params.get("locale")
1142     rendition = str(params.get("rendition") or "original")
1143     if rendition not in {"original", "sd"}:
1144         return {
1145             "status": "failed",
1146             "message": f"Unsupported compile rendition: {rendition}",
1147             "video_id": video_id,
1148         }
1149
1150     notify("resolving")
1151     try:
1152         video, kept, out_name = _resolve_compile(
1153             db, cfg, video_id, segment_ids, params.get("output_filename", "")
1154         )
1155     except _CompileError as e:
1156         return {"status": "failed", "message": e.detail, "video_id": video_id}
1157
1158     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1159     output_dir = (
1160         getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1161     )
1162     # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
1163     # unknown/absent locale keeps the configured default unchanged.
1164     localized_watermark = localize_watermark(stitching.watermark, locale)
1165     encode_profile = _rendition_encode_profile(stitching, rendition)
1166     compiler = _main.VideoCompiler(
1167         output_dir,
1168         begin_padding=stitching.begin_padding,
1169         end_padding=stitching.end_padding,
1170         watermark=localized_watermark,
1171         encode_profile=encode_profile,
1172     )
1173     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1174
1175     notify("stitching")
1176     logger.info(
1177         "[compile] stitching video_id=%s rendition=%s output=%s",
1178         video_id,
1179         rendition,
1180         out_name,
1181     )
1182     local_src = ""
1183     try:
1184         # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1185         # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1186         # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
1187         local_src = storage.acquire_local_input(
1188             video["filepath"], getattr(cfg, "storage", None)

```

```

1189         )
1190         # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
1191         # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
1192         with _LOCAL_COMPUTE_LANE:
1193             out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1194             except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1195                 logger.error(
1196                     "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1197                 )
1198             return {
1199                 "status": "failed",
1200                 "message": f"Highlight stitching failed: {e}",
1201                 "video_id": video_id,
1202             }
1203         finally:
1204             if local_src:
1205                 storage.cleanup_acquired_local_input(
1206                     video["filepath"], local_src, getattr(cfg, "storage", None)
1207                 )
1208
1209         # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
1210         # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
1211         # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
1212         # still falls back to the local file on the same instance).
1213         reel_uri = reel_object_uri(cfg, video_id, out_name)
1214         if reel_uri:
1215             try:
1216                 notify("uploading reel")
1217                 # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
1218                 # project/client) is used, not a fresh default.
1219                 storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1220             except Exception as e: # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
1221                 logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1222
1223         # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1224         # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1225         download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1226         notify("finished")
1227         return {
1228             "status": "success",
1229             "filepath": out_path,
1230             "download_url": download_url,
1231             "video_id": video_id,
1232             "filename": out_name,
1233             "rendition": rendition,
1234         }
1235

```

20. assistant (2026-07-08T06:06:08.451Z)

The working-tree `\_execute\_compile` doesn't match the post-#525 diff (it has `\_rendition`/`\_encode\_profile` code). I need to understand the actual repo state before reviewing ? let me check where HEAD, `origin/master`, and dac6736 sit.

21. user (2026-07-08T06:06:12.296Z)

```

HEAD:
bf97700facc2c7762a4cb2c7c039e7673eebab8b
--- branches containing dac6736 ---
+ perf/521-cheap-watermark
  remotes/origin/HEAD -> origin/master
  remotes/origin/master
--- is dac6736 ancestor of HEAD? ---
NO not ancestor
--- log first-parent from HEAD, 8 ---
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
($9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload
(#518)
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)
115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)
ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker
(#515)
f5c9dd9 chore(ui): re-vendor SPA bundle with the hosted compute-picker cloud default (#516)
ec2d080 fix(serving): fail fast when compute=local forces weightless in-process detection (#511)
20862d7 perf(validators): decode scene_cut/blur/relevance frames in one shared pass (#512)

```

22. assistant (2026-07-08T06:06:28.159Z)

dac6736 (PR #525) sits on branch `perf/521-cheap-watermark`, with its parent = current HEAD `bf97700`. But the diff showed the pre-change `\_execute\_compile` WITHOUT the `rendition` code that HEAD's working tree has. That's a contradiction worth chasing ? it may indicate a stale-base squash merge that reverts a feature. Let me compare the parent vs merged file directly.

23. user (2026-07-08T06:06:31.647Z)

```
=== rendition references in PARENT bf97700 orchestration.py ===
```

```
=== rendition references in MERGED dac6736 orchestration.py ===
```

24. assistant (2026-07-08T06:06:38.223Z)

Neither parent nor merged has `rendition`, yet the on-disk file does. The working tree has diverged from what I think HEAD is. Let me check the live repo state now.

25. user (2026-07-08T06:06:42.259Z)

```

=== git status ===
 M README.md
 M SESSION_HANDOFF.md
 M backend/config/models.py
 M backend/infrastructure/database_jobs.py
 M backend/main.py
 M backend/orchestration.py
 M backend/pipeline/stitcher/compiler.py
 M config.json
 M tests/test_api_endpoints.py
 M tests/test_async_jobs.py
=== HEAD now ===
bf97700facc2c7762a4cb2c7c039e7673eebab8b
=== does disk match HEAD for orchestration.py? ===
warning: in the working copy of 'backend/orchestration.py', LF will be replaced by CRLF the next
time Git touches it
 backend/orchestration.py | 30 ++++++
 1 file changed, 30 insertions(+)
=== rendition on disk? ===
15
=== which commits touch rendition in orchestration ===

```

26. assistant (2026-07-08T06:06:54.131Z)

The working tree has unrelated uncommitted WIP (a `rendition`/cheap-watermark change) layered on `bf97700` ? not part of #525. The authoritative review target is the `dac6736` tree. Let me extract the post-#525 source of the key files to review the invariants properly.

27. user (2026-07-08T06:07:00.828Z)

```
total 220
drwxr-xr-x 1 avidu 197609    0 Jul  8 11:37 .
drwxr-xr-x 1 avidu 197609    0 Jul  8 11:36 ..
-rw-r--r-- 1 avidu 197609 25174 Jul  8 11:37 backend_compute_cloud_run.py
-rw-r--r-- 1 avidu 197609 36917 Jul  8 11:37 backend_config_models.py
-rw-r--r-- 1 avidu 197609  2852 Jul  8 11:37 backend_config_placement.py
-rw-r--r-- 1 avidu 197609 71951 Jul  8 11:36 backend_orchestration.py
-rw-r--r-- 1 avidu 197609 28017 Jul  8 11:36 backend_pipeline_stitcher_compiler.py
-rw-r--r-- 1 avidu 197609 40742 Jul  8 11:37 backend_worker__main__.py
```

28. assistant (2026-07-08T06:07:04.500Z)

Now let me read the post-#525 `\_stitch\_reel`, `reel\_object\_uri`, and `\_LOCAL\_COMPUTE\_LANE` from the merged tree.

29. user (2026-07-08T06:07:06.056Z)

```
1100 def _resolve_compile(
1101     db, cfg, video_id: str, segment_ids: List[int], output_filename: str
1102 ):
1103     """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
1104     resolve, since a
1105     row could change between submit and run). Verifies the video exists, the requested
1106     segments match
1107     and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known
1108     shot_count are exempt
1109     so manual/legacy picks can't silently vanish), and the output name is traversal-
1110     safe. Raises
1111     ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
1112     safe_out_name)``."""
1113     video = db.get_video(video_id)
1114     if not video:
1115         raise _CompileError(404, "Indexed video record not found.")
1116     # Reject path traversal / absolute names (os.path.join would otherwise write
1117     anywhere on disk).
1118     try:
1119         out_name = safe_output_filename(output_filename)
1120     except ValueError as e:
1121         raise _CompileError(400, str(e)) from e
1122
1123     all_segments = db.query_play_segments(video_id, {})
1124     matched = [s for s in all_segments if s["id"] in segment_ids]
1125     if not matched:
1126         raise _CompileError(400, "No matching play segments found to stitch.")
1127
1128     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1129     kept = []
1130     for s in matched:
1131         meta = s.get("metadata") or {}
1132         sc = meta.get("shot_count") if isinstance(meta, dict) else None
1133         try:
1134             if sc is not None and int(sc) < stitching.min_shot_count:
1135                 continue
1136         except (TypeError, ValueError):
1137             pass # unparseable count -> treat as unknown, keep
1138         kept.append(s)
1139     if len(kept) < len(matched):
1140         logger.info(
```

```

1135         "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
floor.",
1136         stitching.min_shot_count,
1137         len(matched) - len(kept),
1138         len(matched),
1139     )
1140     if not kept:
1141         raise _CompileError(
1142             400,
1143             f"All {len(matched)} matching segments fall below "
1144             f"stitching.min_shot_count={stitching.min_shot_count}.",
1145         )
1146     return video, kept, out_name
1147
1148
1149     def reel_object_uri(cfg, video_id: str, name: str) -> str:
1150         """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
1151         (#463), or "" when
1152         the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
1153         compile
1154         mirror-upload and the /api/highlights redirect:
1155         ``<output_root>/highlights/<video_id>/<name>``. """
1156         root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
1157         if not root or "://" not in root:
1158             return ""
1159         return f"{root.rstrip('/')}/highlights/{video_id}/{name}"
1160
1161
1162     def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
1163         """The active storage settings dict (per-backend config for the storage layer ? S3
1164         endpoint/region,
1165         GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
1166         default backend
1167         isn't silently constructed as a fresh default. """
1168         sc = getattr(cfg, "storage", None)
1169         return sc.model_dump() if sc is not None else None
1170
1171
1172     def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
1173         """A short-lived signed URL for a reel mirrored to the output bucket, so
1174         /api/highlights can
1175         307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
1176         ephemeral local
1177         /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
1178         config for BOTH
1179         the existence check and the signing (extracted here to keep backend/main.py lean ?
1180         P23/P24). """
1181         reel_uri = reel_object_uri(cfg, video_id, name)
1182         if not reel_uri:
1183             return None
1184         settings = _storage_settings(cfg)
1185         if storage.exists(reel_uri, config=settings):
1186             return storage.signed_url(reel_uri, config=settings, method="GET")
1187         return None
1188
1189
1190     def _compile_wait_message(elapsed_sec: float) -> str:
1191         """The live job.progress line while an L4 compile/stitch is in flight (#521; mirrors
1192         ``_wait_progress_message``). Minute-granular; under a minute reads as cold-start. """
1193         minutes = int(elapsed_sec) // 60
1194         if minutes < 1:
1195             return "stitching on cloud GPU (NVENC) ? starting up"
1196         return f"stitching on cloud GPU (NVENC) ? {minutes} min elapsed"

```



```

1190     def _stitch_reel(
1191         cfg,
1192         stitching,
1193         video_id: str,
1194         source_ref: str,
1195         time_pairs: List[Tuple[float, float]],
1196         out_name: str,
1197         locale: Optional[str],
1198         notify,
1199     ) -> Dict[str, Any]:
1200         """Stitch + mirror-upload core, shared by the control-plane compile handler AND the
L4 worker
1201         (#521). ``time_pairs`` are already RESOLVED (the caller filtered via
``_resolve_compile``), so this
1202         needs NO control-plane DB ? the worker reads them from the staged spec. Builds the
VideoCompiler
1203         with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the source,
runs the
1204         trim/watermark/concat, mirrors the reel to
``<output_root>/highlights/<video_id>/<name>``, and
1205         returns the ``{status, filepath, download_url, video_id, reel_uri}`` the SPA polls
for. Returns a
1206         ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg
failure)."""
1207         import backend.main as _main # call-time seam: VideoCompiler stays patchable on
backend.main
1208
1209         output_dir = (
1210             getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1211         )
1212         # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
1213         # unknown/absent locale keeps the configured default unchanged.
1214         localized_watermark = localize_watermark(stitching.watermark, locale)
1215         compiler = _main.VideoCompiler(
1216             output_dir,
1217             begin_padding=stitching.begin_padding,
1218             end_padding=stitching.end_padding,
1219             watermark=localized_watermark,
1220             encoder=getattr(stitching, "encoder", "libx264"),
1221         )
1222
1223         notify("stitching")
1224         local_src = ""
1225         try:
1226             # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1227             # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1228             # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
1229             local_src = storage.acquire_local_input(
1230                 source_ref, getattr(cfg, "storage", None)
1231             )
1232             # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
1233             # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
1234             with _LOCAL_COMPUTE_LANE:
1235                 out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1236             except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1237                 logger.error(
1238                     "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1239                 )

```

```

1240         return {
1241             "status": "failed",
1242             "message": f"Highlight stitching failed: {e}",
1243             "video_id": video_id,
1244         }
1245     finally:
1246         if local_src:
1247             storage.cleanup_acquired_local_input(
1248                 source_ref, local_src, getattr(cfg, "storage", None)
1249             )
1250
1251     # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
1252     # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
1253     # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
1254     # still falls back to the local file on the same instance). On the L4 worker the
mirror is how the
1255     # reel reaches the bucket at all (the worker's local file is discarded when the Job
exits).
1256     reel_uri = reel_object_uri(cfg, video_id, out_name)
1257     if reel_uri:
1258         try:
1259             notify("uploading reel")
1260             # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
1261             # project/client) is used, not a fresh default.
1262             storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1263             except Exception as e: # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
1264             logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1265
1266     # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1267     # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1268     download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1269     notify("finished")
1270     return {
1271         "status": "success",
1272         "filepath": out_path,
1273         "download_url": download_url,
1274         "video_id": video_id,

```

30. assistant (2026-07-08T06:07:40.931Z)

Let me read the full compiler `compile\_highlights` including the concat step and `\_trim\_one\_segment` from the merged tree.

31. user (2026-07-08T06:07:44.232Z)

```

300
301     def _trim_one_segment(
302         self,
303         idx: int,
304         raw_start: Union[int, float, str],
305         raw_end: Union[int, float, str],
306         segments: List[Tuple[float, float]],
307         video_duration: float,
308         tmp_dir: str,
309         raw_video_path: str,
310         watermark_filter: Optional[str],
311         video_codec_args: Optional[List[str]] = None,
312     ) -> Tuple[Optional[str], Optional[Tuple[int, float, float, str]]]:
313         """Trim a single segment into its own clip file (extracted verbatim from the

```

```

314         per-segment body of compile_highlights). Returns (clip_path, skip_record): on
315         success the clip path and None; on a skip (invalid/out-of-range/ffmpeg failure)
316         None and the (idx, start, end, reason) record to record in skipped_segments."""
317         # Normalize timestamps to float seconds ? handles float, int,
318         # plain numeric strings, and colon-separated formats like "MM:SS"
319         start = self._parse_time(raw_start)
320         end = self._parse_time(raw_end)
321         segment_label = (
322             f"Segment #{idx + 1}/{len(segments)} [{start:.2f}s - {end:.2f}s]"
323         )
324
325         # Validate the segment times
326         if start < 0:
327             logger.warning(
328                 f"{segment_label}: start_time is negative ({start:.2f}s). Clamping to
329                 0."
330             )
331             start = 0.0
332
333         if start >= end:
334             logger.error(
335                 f"{segment_label}: start_time ({start:.2f}s) >= end_time ({end:.2f}s).
336                 Skipping invalid segment."
337             )
338             return None, (idx, start, end, "start >= end")
339
340         # Check if segment extends beyond video duration
341         if video_duration > 0:
342             if start >= video_duration:
343                 logger.error(
344                     f"{segment_label}: start_time ({start:.2f}s) is beyond the video
345                     duration ({video_duration:.2f}s). "
346                     f"This segment cannot be extracted ? the video ran out. Skipping."
347                 )
348                 return None, (idx, start, end, "start beyond video end")
349
350             if end > video_duration:
351                 original_end = end
352                 end = video_duration
353                 logger.warning(
354                     f"{segment_label}: end_time ({original_end:.2f}s) exceeds video
355                     duration ({video_duration:.2f}s). "
356                     f"Clamping end_time to {video_duration:.2f}s. The segment may be
357                     truncated."
358                 )
359
360         clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")
361
362         # Precise sub-second trim using fast re-encoding to preserve stream headers
363         padded_start = max(0.0, start - self.begin_padding)
364         padded_end = end + self.end_padding
365
366         # Clamp padded_end to video duration if known
367         if video_duration > 0 and padded_end > video_duration:
368             padded_end = video_duration
369             logger.info(
370                 f"{segment_label}: Padded end ({end + self.end_padding:.2f}s) exceeds
371                 video duration. "
372                 f"Clamping to {video_duration:.2f}s (end padding partially applied)."
373             )
374
375         trim_duration = padded_end - padded_start
376         logger.info(
377             f"Trimming {segment_label} -> [{padded_start:.3f}s - {padded_end:.3f}s]
378             (duration: {trim_duration:.2f}s)"
379         )

```

```

372         )
373
374         # Default to libx264 (the historical settings) when a caller invokes
_trim_one_segment
375         # directly without resolving an encoder ? keeps this method's old behaviour
intact.
376         codec_args = (
377             video_codec_args
378             if video_codec_args is not None
379             else ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"]
380         )
381         base_cmd = [
382             ffmpeg_bin(),
383             "-y",
384             "-ss",
385             str(round(padded_start, 3)),
386             "-to",
387             str(round(padded_end, 3)),
388             "-i",
389             raw_video_path,
390             *codec_args,
391             "-c:a",
392             "aac",
393             "-b:a",
394             "128k",
395         ]
396         vf_args = ["-vf", watermark_filter] if watermark_filter else []
397
398         # Try with the watermark first; if that encode fails (e.g. a bad font/logo
399         # path), retry once WITHOUT it so a branding misconfig can never nuke the
400         # whole reel. Clips stay codec-identical either way, so concat -c copy holds.
401         attempts = [vf_args, []] if vf_args else [[]]
402         produced = False
403         last_exc: Optional[subprocess.CalledProcessError] = None
404         cmd = base_cmd + [clip_path]
405         timeout = ffmpeg_timeout_sec()
406         for attempt_vf in attempts:
407             cmd = base_cmd + attempt_vf + [clip_path]
408             try:
409                 subprocess.run(
410                     cmd, capture_output=True, check=True, timeout=timeout
411                 )
412             except subprocess.TimeoutExpired:
413                 reason = f"ffmpeg timeout after {timeout_label(timeout)}"
414                 logger.error(f"{segment_label}: FFmpeg trim timed out.")
415                 print(f"  Command: {' '.join(cmd)}")
416                 print(f"  {reason}")
417                 return None, (idx, start, end, reason)
418             except subprocess.CalledProcessError as e:
419                 last_exc = e
420                 if attempt_vf and vf_args:
421                     _err = (
422                         e.stderr.decode(errors="replace").strip()[-300:]
423                         if getattr(e, "stderr", None)
424                         else ""
425                     )
426                     logger.warning(
427                         f"{segment_label}: watermark encode failed; retrying without it.
428 "
429                         f"ffmpeg: {_err or '(no stderr captured)'}"
430                     )
431                     continue
432                 break
433         if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
434             produced = True

```

```

434         if not attempt_vf and vf_args:
435             logger.warning(
436                 f"{segment_label}: encoded WITHOUT the watermark (the watermark
filter failed)."
437             )
438             break
439         last_exc = None # ffmpeg succeeded but produced nothing
440         break
441
442     if produced:
443         return clip_path, None
444     elif last_exc is not None:
445         stderr_msg = last_exc.stderr.decode(errors="replace").strip()
446         logger.error(f"{segment_label}: FFmpeg trim failed.")
447         print(f"  Command: {' '.join(cmd)}")
448         print(
449             f"    FFmpeg stderr: {stderr_msg[-500:]}")
450         ) # Last 500 chars to avoid flooding
451         return None, (idx, start, end, f"ffmpeg error: {stderr_msg[-200:]}")
452     else:
453         logger.error(
454             f"{segment_label}: FFmpeg exited successfully but output clip is missing
or empty. Skipping."
455         )
456         return None, (idx, start, end, "empty output clip")
457
458     def compile_highlights(
459         self,
460         raw_video_path: str,
461         segments: List[Tuple[float, float]],
462         output_filename: str,
463     ) -> str:
464         """
465         Extracts selected segments from a raw video and stitches them together
466         into a seamless highlights file, applying end_padding to prevent abrupt endings.
467
468         .. warning::
469             This method uses a two-stage codec contract: per-clip re-encodes must use
470             IDENTICAL encoder settings (codec, preset, CRF, pixel format) so the final
471             concat can stream-copy (``-c copy``) without re-encoding. Changing the
472             clip encode settings without updating the concat assumptions will produce
473             broken output.
474         """
475         if not segments:
476             raise ValueError("No highlight segments provided to compile.")
477
478         # Never stitch the same footage twice: collapse overlapping/duplicate ranges
479         # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
480         original_count = len(segments)
481         segments = self._merge_overlapping(segments)
482         if len(segments) < original_count:
483             logger.info(
484                 "Merged %d overlapping/duplicate segment(s) before stitching: %d ->
%d.",
485                 original_count - len(segments),
486                 original_count,
487                 len(segments),
488             )
489         if not segments:
490             raise ValueError("No valid (start < end) highlight segments after merge.")
491
492         if not os.path.exists(raw_video_path):
493             raise FileNotFoundError(
494                 f"[COMPILER ERROR] Source video file not found: {raw_video_path}"
495             )

```

```

496
497     # Defense-in-depth: never let the output name escape output_dir (the API also
498     # validates this at the request boundary).
499     output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
500
501     # Probe video duration so we can detect out-of-range segments
502     video_duration = self._get_video_duration(raw_video_path)
503     if video_duration > 0:
504         logger.info(f"Source video duration: {video_duration:.2f}s")
505     else:
506         logger.warning(
507             "Video duration unknown. Cannot validate segment boundaries against
video length."
508         )
509
510     # We will create temporary clips and stitch them using FFmpeg concat demuxer
511     temp_clips = []
512     skipped_segments = []
513
514     # Create a temp directory within output_dir
515     with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
516         logger.info(
517             f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)..."
518         )
519         # Build the (optional) watermark filtergraph once; it is applied during
every
520         # per-clip re-encode so the mark is baked into each clip identically.
521         watermark_filter = self._watermark_filter(tmp_dir)
522         if watermark_filter:
523             logger.info("[watermark] applying branding overlay to each clip.")
524         # Resolve the video encoder ONCE (probe NVENC / fall back) so every clip
shares one codec
525         # and the final concat stays -c copy (#521).
526         video_codec_args, chosen_encoder = self._select_encoder()
527         if chosen_encoder != "libx264":
528             logger.info("[compile] encoding clips with %s (hardware).",
chosen_encoder)
529         for idx, (raw_start, raw_end) in enumerate(segments):
530             clip_path, skip_record = self._trim_one_segment(
531                 idx,
532                 raw_start,
533                 raw_end,
534                 segments,
535                 video_duration,
536                 tmp_dir,
537                 raw_video_path,
538                 watermark_filter,
539                 video_codec_args,
540             )
541             if clip_path is not None:
542                 temp_clips.append(clip_path)
543             if skip_record is not None:
544                 skipped_segments.append(skip_record)
545
546         # Report summary of skipped segments
547         if skipped_segments:
548             logger.info(
549                 f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:"
550             )
551             for seg_idx, seg_start, seg_end, reason in skipped_segments:
552                 logger.info(
553                     f"    - Segment #{seg_idx + 1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}"

```

```

554         )
555         logger.info("")
556
557     if not temp_clips:
558         raise RuntimeError(
559             f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "

```

32. assistant (2026-07-08T06:07:59.660Z)

Now the concat step itself.

33. user (2026-07-08T06:08:02.971Z)

```

557     if not temp_clips:
558         raise RuntimeError(
559             f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
560             f"All {len(segments)} segments were skipped. "
561             f"Check the errors above for details. "
562             f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
563         )
564
565     logger.info(
566         f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch..."
567     )
568
569     # Create a list file for FFmpeg concat demuxer
570     list_file_path = os.path.join(tmp_dir, "clips.txt")
571     # encoding pinned: a non-ASCII clip path would crash on Windows (cp1252
default);
572     # ffmpeg's concat demuxer reads the list as UTF-8 on every OS.
573     with open(list_file_path, "w", encoding="utf-8") as f:
574         for clip in temp_clips:
575             # The concat demuxer resolves relative entries against the list
file's
576             # directory. Use absolute paths so a temp-dir clip does not become
577             # output/tmp.../output/tmp.../clip_0.mp4 on Windows.
578             safe_path = Path(clip).resolve().as_posix().replace("'", "'\\'")
579             f.write(f"file '{safe_path}'\n")
580
581     logger.info("Stitching clips together...")
582     # Stitch them together without re-encoding (since they were encoded with
identical settings)
583     concat_cmd = [
584         ffmpeg_bin(),
585         "-y",
586         "-f",
587         "concat",
588         "-safe",
589         "0",
590         "-i",
591         list_file_path,
592         "-c",
593         "copy",
594         output_path,
595     ]
596
597     try:
598         timeout = ffmpeg_timeout_sec()
599         subprocess.run(
600             concat_cmd, capture_output=True, check=True, timeout=timeout
601         )
602
603     # Final verification

```

```

604         if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
605             raise RuntimeError(
606                 f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
607                 f"but output file is missing or empty: {output_path}"
608             )
609
610         output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
611         logger.info(
612             f"Dynamic highlight reel saved: {output_path} ({output_size_mb:.1f}
613         MB)"
614         )
615         return output_path
616     except subprocess.CalledProcessError as e:
617         stderr_msg = e.stderr.decode(errors="replace").strip()
618         raise RuntimeError(
619             f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
620             f"Command: {' '.join(concat_cmd)}\n"
621             f"FFmpeg stderr: {stderr_msg[-500:]}"
622         ) from e
623     except subprocess.TimeoutExpired as e:
624         raise RuntimeError(
625             f"[COMPILER ERROR] FFmpeg concatenation timed out after
626             {timeout_label(timeout)}.\n"
627             f"Command: {' '.join(concat_cmd)}"
628         ) from e

```

34. assistant (2026-07-08T06:08:37.124Z)

The concat is ``-c copy`` and all clips share one resolved encoder ? that invariant holds. Now let me scrutinize the validation-delegation parity: ``_will_detect_on_cloud`` must agree with the actual dispatch decision in ``_maybe_dispatch_remote``, or nobody validates. Let me read the full ``_maybe_dispatch_remote`` and ``_execute_processing`` validation block from the merged tree.

35. user (2026-07-08T06:08:40.561Z)

```

500     choice = (req.compute or "").strip().lower()
501     if choice and choice not in ("local", "cloud"):
502         logger.warning(
503             "[API] ignoring unknown compute=%r (expected 'local'|"cloud'); using server
504             default.",
505             req.compute,
506         )
507     choice = ""
508     if choice == "local":
509         # A hosted cloud-serving control-plane bakes NO detector weights into its image
510         # (only the
511         # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
512         trajectory detector
513         # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
514         a cryptic
515         # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail FAST
516         with an
517         # actionable message instead of that silent trap. A truly-local box (not cloud-
518         capable), or a
519         # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it runs
520         in-process
521         # below exactly as before (the weights-present check is the escape hatch).
522         if (
523             (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
524             and _cloud_available(cfg)
525             and not _local_detector_weights_present(cfg, seg_name)
526         ):
527             return _failed(
528                 f"this hosted server can't run detection locally ? the '{seg_name}'

```



```

model weights "
522             "live only on the cloud worker, not on the control-plane. Choose 'Run in
cloud'.",
523             False,
524         )
525         return None # explicit "run on my machine" ? in-process regardless of the
server's target
526         if choice == "cloud":
527             if not _cloud_available(cfg):
528                 return _failed(
529                     "cloud-serving was requested but this server isn't configured for it "
530                     "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
531                     False,
532                 )
533             compute = get_compute_backend(cfg, target_override="cloud_run")
534         else:
535             # No explicit picker ? consult phase_placement.segment (#521). Its legacy
default is
536             # cloud_run_l4 IFF compute_target==cloud_run, so this preserves today's
behaviour; setting
537             # phase_placement.segment relocates DETECT by config alone (e.g. force it back
on the
538             # control-plane, or onto the L4 on a default-local box wired for cloud).
539             if (
540                 resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
541                 and _cloud_available(cfg)
542             ):
543                 compute = get_compute_backend(cfg, target_override="cloud_run")
544             else:
545                 compute = None
546         if compute is None:
547             return None # default-OFF / placed on control_plane ? the in-process segmenter
below
548
549         storage_cfg = getattr(cfg, "storage", None)
550
551         # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
552         try:
553             input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
554         except ValueError as e:
555             return _failed(
556                 f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
557             )
558         if input_ref.backend == "local":
559             # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
560             # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
561             # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
562             # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
563             # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
564             # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
at its
565             # default, while a bare/local-looking root can't stage no matter what
input_backend says
566             # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
567             # gives us nothing to build the staged URI under).
568             input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
569             try:

```

```

570         root_backend = (
571             storage.parse_uri(input_root).backend if input_root else "local"
572         )
573     except ValueError: # unsupported scheme in input_root ? nowhere real to stage
574         to
575         root_backend = "local"
576     if root_backend == "local":
577         return _failed(
578             "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
579             storage.input_root to "
580             "stage local uploads into; a path local to this machine can't be read by
581             the Cloud "
582             "Run job.",
583             False,
584         )
585     local_src = input_ref.key # resolved local path of the upload
586     staged_uri =
587     f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
588     try:
589         notify("staging upload to bucket")
590         # Thread the active storage settings so backend-specific config survives (S3
591         endpoint_url/
592         # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh
593         default client.
594         storage.put(
595             local_src,
596             staged_uri,
597             config=(
598                 storage_cfg.model_dump() if storage_cfg is not None else None
599             ),
600         )
601     except Exception as e: # noqa: BLE001 ? a staging failure is a clean job
602         failure, not a 500
603         logger.error(
604             "[API] cloud-serving: failed to stage %s -> %s: %s",
605             local_src,
606             staged_uri,
607             e,
608             exc_info=True,
609         )
610     return _failed(
611         f"cloud-serving: failed to stage the uploaded file to {input_root}:
612         {e}",
613         False,
614     )
615     # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
616     the L4 job
617     # DETECTs from it, and a later /api/compile re-fetches the same source (the
618     local upload is
619     # gone once this instance recycles). add_video is an UPSERT ? it only updates
620     filepath.
621     req.video_path = staged_uri
622     input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
623     db.add_video(
624         video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
625     )
626     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
627     if not out_root:
628         return _failed(
629             "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
630             bucket) "
631             "? that's where the Cloud Run job writes the result.",
632             False,
633         )

```

```

623     notify("dispatching (cloud_run)")
624     logger.info(
625         "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
626         req.video_path,
627         out_root,
628     )
629     # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
630     # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
631     # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
632     # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
633     overrides = []
634     idx_cfg = getattr(cfg, "indexing", None)
635     sk = getattr(idx_cfg, "skip_ai_handoff", None)
636     if sk is not None:
637         overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
638     # Forward the resolved WASB serving knobs so the PRESET governs the worker
(#506/#507). The
639     # worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving
preset itself ?
640     # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
model
641     # defaults (cpu, batch 8) regardless of what the control-plane resolved.
decode_backend is
642     # cuda-only (validator-enforced) and requires a worker image that understands
--decode-backend
643     # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
parity default).
644     wasb_cfg = getattr(idx_cfg, "wasb", None)
645     if wasb_cfg is not None:
646         decode_backend = getattr(wasb_cfg, "decode_backend", None)
647         if decode_backend:
648             overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
649         batch_size = getattr(wasb_cfg, "batch_size", None)
650         if batch_size:
651             overrides.append(f"indexing.wasb.batch_size={batch_size}")
652         prefetch = getattr(wasb_cfg, "prefetch_batches", None)
653         if prefetch:
654             overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
655     # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
656     # gate ? it already ran the validators with its resolved config just above (only a
PASS reaches
657     # here; a failure returned "failed" before dispatch). A worker re-run would decode
the video a
658     # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
659     # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
660     # then worker-rejected at its default 20.0 after a successful GPU dispatch).
661     # Who validates? phase_placement.validation (#521). DEFAULT (control_plane): the
control-plane is
662     # the authoritative gate ? it already validated with its resolved config (only a
PASS reaches
663     # here), so the worker SKIPS its redundant re-validation (#513). If validation is
placed on the
664     # L4 (cloud_run_l4), the control-plane delegated it (it did NOT run the validators ?
see
665     # _execute_processing), so the worker must NOT skip: it becomes the gate.
666     worker_skips_validation = (
667         resolve_phase_placement(cfg, PHASE_VALIDATION) != PLACE_CLOUD_L4
668     )

```

```

669         spec = ComputeJobSpec(
670             video_uri=req.video_path,
671             out_uri=out_root,
672             segmenter=seg_name,
673             config_overrides=tuple(overrides),
674             skip_validation=worker_skips_validation,
675         )
676     try:
677         # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
678         # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
679         # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which

```

36. assistant (2026-07-08T06:12:39.628Z)

Let me verify the compiler imports (the diff added ``Dict`/`Tuple`` type usage and ``subprocess`/`ffmpeg_timeout_sec``) and check ``_LOCAL_COMPUTE_LANE``'s definition and the worker ``cmd_compile`` fallback around `output_root`.

37. user (2026-07-08T06:12:43.336Z)

```

=== compiler.py imports (dac6736) ===
import logging
import os
import subprocess
import tempfile
from pathlib import Path
from typing import Any, Dict, List, Optional, Tuple, Union

from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
    ffmpeg_bin,
    ffmpeg_timeout_sec,
    ffprobe_bin,
    ffprobe_timeout_sec,
    timeout_label,
)

logger = logging.getLogger(__name__)

```

Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the over renders out-of-the-box without a system fontconfig setup (a common gap on Windows ``font='Sans'`` fails). Overridable via `stitching.watermark.font_path`. This module lives at `backend/pipeline/stitcher/`, so `parents[2] == backend/`.

```

_BUNDLED_FONT = (
    Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-Regular.ttf"
)

=== _LOCAL_COMPUTE_LANE def (dac6736 orchestration) ===
62:_LOCAL_COMPUTE_LANE = threading.BoundedSemaphore(1)
982:         # Serialize the CPU/GPU-heavy in-process detection (see _LOCAL_COMPUTE_LANE). A
remote
984:         with _LOCAL_COMPUTE_LANE:
1233:         # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
1234:         with _LOCAL_COMPUTE_LANE:

=== ffmpeg_timeout_sec source ===
backend/utils/ffmpeg.py:62:def ffmpeg_timeout_sec() -> Optional[float]:

```

38. user (2026-07-08T06:12:52.901Z)

```

55
56
57     def ffprobe_timeout_sec() -> Optional[float]:
58         """Bound metadata probes. Set ``RALLY_FFPROBE_TIMEOUT_SEC=0`` to wait forever."""
59         return _timeout_seconds("RALLY_FFPROBE_TIMEOUT_SEC", DEFAULT_FFPROBE_TIMEOUT_SEC)
60
61
62     def ffmpeg_timeout_sec() -> Optional[float]:
63         """Bound transcode/extract/concat calls. Set ``RALLY_FFMPEG_TIMEOUT_SEC=0`` to wait
64         forever."""
65         return _timeout_seconds("RALLY_FFMPEG_TIMEOUT_SEC", DEFAULT_FFMPEG_TIMEOUT_SEC)
66
67     def timeout_label(seconds: Optional[float]) -> str:
68         return "forever" if seconds is None else f"{seconds:g}s"
69
70
71     def ffmpeg_available() -> bool:
72         return shutil.which(ffmpeg_bin()) is not None
73
74
75     def ffprobe_available() -> bool:
76         return shutil.which(ffprobe_bin()) is not None
77
78
79     def install_hint() -> str:
80         """The OS-appropriate one-line install command (mirrors scripts/doctor.py)."""
81         system = platform.system()
82         if system == "Windows":
83             return "winget install Gyan.FFmpeg"
84         if system == "Darwin":

```

39. assistant (2026-07-08T06:13:00.709Z)

Let me examine the new tests to see which failure modes are covered and confirm my candidate findings aren't already guarded.

40. user (2026-07-08T06:13:03.000Z)

```

=== test_compiler.py (dac6736) encoder tests ===
"""VideoCompiler coverage (previously zero ? the user-facing highlight-reel output path,
where the f-string SyntaxError lived undetected). ffmpeg/ffprobe are mocked."""

import os
import subprocess
from unittest.mock import MagicMock, patch

import pytest

from backend.pipeline.stitcher.compiler import VideoCompiler

def test_parse_time_formats():
    f = VideoCompiler._parse_time
    assert f(12.5) == 12.5
    assert f("12.5") == 12.5
    assert f("1:30") == 90.0
    assert f("1:00:00") == 3600.0
    assert f("garbage") == 0.0

def test_merge_overlapping_collapses_duplicates_and_overlaps():
    m = VideoCompiler._merge_overlapping
    # Exact duplicates + near-duplicates (the "every rally twice" re-run bug) collapse to one
    each.

```

```

assert m([(28.2, 34.5), (28.5, 34.5), (40.5, 45.5), (40.5, 45.5)]) == [
    (28.2, 34.5),
    (40.5, 45.5),
]
# Unsorted input is sorted; an overlapping range extends the previous.
assert m([(100.0, 110.0), (0.0, 5.0), (104.0, 120.0)]) == [
    (0.0, 5.0),
    (100.0, 120.0),
]
# Genuinely distinct (gapped) rallies are preserved, not merged.
assert m([(0.0, 5.0), (6.0, 9.0)]) == [(0.0, 5.0), (6.0, 9.0)]
# Degenerate rows (end <= start) are dropped; string timestamps are accepted.
assert m([("0:10", "0:20"), (30.0, 30.0), (31.0, 29.0)]) == [(10.0, 20.0)]

def test_compile_dedups_before_stitching(tmp_path):
    """A segment list with every rally twice must yield ONE clip per unique rally."""
    comp = VideoCompiler(str(tmp_path / "out"))
    raw = tmp_path / "raw.mp4"
    raw.write_bytes(b"x")
    trims = []

    def fake_run(cmd, *a, **k):
        if cmd[0] == "ffprobe":
            return MagicMock(stdout="100.0\n", returncode=0)
        out = cmd[-1]
        if isinstance(out, str) and out.endswith(".mp4"):
            open(out, "wb").write(b"clip")
            # count per-clip trims (the concat list file input is not an .mp4 output)
            if "-i" in cmd and cmd.index("-i") + 1 == str(raw):
                trims.append(cmd)
        return MagicMock(returncode=0, stdout=b"", stderr=b"")

    dup_segments = [(0.0, 5.0), (0.2, 5.0), (10.0, 15.0), (10.0, 15.0), (20.0, 25.0)]
    with patch.object(subprocess, "run", side_effect=fake_run):
        comp.compile_highlights(str(raw), dup_segments, "reel.mp4")
    # 5 input segments collapse to 3 unique rallies -> 3 per-clip trims, not 5.
    assert len(trims) == 3, f"expected 3 deduped trims, got {len(trims)}"

def test_compile_basenames_output_and_stitches(tmp_path):
    comp = VideoCompiler(str(tmp_path / "out"))
    raw = tmp_path / "raw.mp4"
    raw.write_bytes(b"x")

    calls = []

    def fake_run(cmd, *a, **k):
        calls.append(cmd)
        # Create whatever output file ffmpeg/ffprobe was told to write so existence checks pass.
        if cmd[0] == "ffprobe":
            return MagicMock(stdout="100.0\n", returncode=0)
        out = cmd[-1]
        if isinstance(out, str) and out.endswith(("mp4",)):
            open(out, "wb").write(b"clip")
        return MagicMock(returncode=0, stdout=b"", stderr=b"")

    with patch.object(subprocess, "run", side_effect=fake_run):
        # Traversal in the name must be stripped (defense-in-depth basename).
        out_path = comp.compile_highlights(
            str(raw), [(0.0, 5.0), (10.0, 15.0)], "../reel.mp4"
        )

    assert os.path.basename(out_path) == "reel.mp4"
    assert os.path.dirname(out_path).endswith("out")

```

```
# Per-clip trims + one concat = at least 3 ffmpeg invocations (plus an ffprobe).
```

41. user (2026-07-08T06:13:12.915Z)

```
=== test_compiler.py encoder-selection tests (tail) ===
# Per-clip trims + one concat = at least 3 ffmpeg invocations (plus an ffprobe).
assert sum(1 for c in calls if c[0] == "ffmpeg") >= 3

def test_compile_concat_timeout_raises_clean_error(tmp_path):
    comp = VideoCompiler(str(tmp_path / "out"))
    raw = tmp_path / "raw.mp4"
    raw.write_bytes(b"x")

    def fake_run(cmd, *a, **k):
        if cmd[0] == "ffprobe":
            return MagicMock(stdout="100.0\n", returncode=0)
        if "concat" in cmd:
            raise subprocess.TimeoutExpired(cmd, k.get("timeout"))
        out = cmd[-1]
        if isinstance(out, str) and out.endswith(".mp4"):
            open(out, "wb").write(b"clip")
        return MagicMock(returncode=0, stdout=b"", stderr=b"")

    with patch.object(subprocess, "run", side_effect=fake_run):
        with pytest.raises(RuntimeError, match="timed out after 1800s"):
            comp.compile_highlights(str(raw), [(0.0, 5.0)], "reel.mp4")

def test_concat_list_uses_absolute_clip_paths(tmp_path):
    """Regression for Windows concat failure:

    ffmpeg resolves relative paths inside clips.txt relative to clips.txt itself. If
    clips.txt lives in output/tmp123 and contains output/tmp123/clip_0.mp4, ffmpeg
    tries output/tmp123/output/tmp123/clip_0.mp4. The list must contain absolute
    clip paths.
    """
    comp = VideoCompiler(str(tmp_path / "out"))
    raw = tmp_path / "raw.mp4"
    raw.write_bytes(b"x")
    concat_list = ""

    def fake_run(cmd, *a, **k):
        nonlocal concat_list
        if cmd[0] == "ffprobe":
            return MagicMock(stdout="100.0\n", returncode=0)
        if "-f" in cmd and "concat" in cmd:
            list_path = cmd[cmd.index("-i") + 1]
            concat_list = open(list_path, encoding="utf-8").read()
        out = cmd[-1]
        if isinstance(out, str) and out.endswith(".mp4"):
            open(out, "wb").write(b"clip")
        return MagicMock(returncode=0, stdout=b"", stderr=b"")

    with patch.object(subprocess, "run", side_effect=fake_run):
        comp.compile_highlights(str(raw), [(0.0, 5.0)], "reel.mp4")

    line = concat_list.strip()
    assert line.startswith("file ")
    clip_path = line.removeprefix("file ").removesuffix("")
    assert os.path.isabs(clip_path)
    assert "clip_0.mp4" in clip_path

def test_compile_raises_on_no_segments(tmp_path):
```

```

comp = VideoCompiler(str(tmp_path / "out"))
raw = tmp_path / "raw.mp4"
raw.write_bytes(b"x")
with pytest.raises(ValueError):
    comp.compile_highlights(str(raw), [], "reel.mp4")

def test_compile_raises_when_source_missing(tmp_path):
    comp = VideoCompiler(str(tmp_path / "out"))
    with pytest.raises(FileNotFoundError):
        comp.compile_highlights(str(tmp_path / "nope.mp4"), [(0.0, 5.0)], "reel.mp4")

```

----- encoder selection (#521 ? NVENC)

```

def test_select_encoder_default_is_libx264(tmp_path):
    """Default encoder = libx264 (CPU) with the historical settings ? no probe, works
    everywhere."""
    comp = VideoCompiler(str(tmp_path / "out"))
    args, chosen = comp._select_encoder()
    assert chosen == "libx264"
    assert args == ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"]

def test_select_encoder_nvenc_when_available(tmp_path, monkeypatch):
    """h264_nvenc resolves to the hardware encoder args when the probe reports it runnable (the
    L4)."""
    from backend.pipeline.stitcher import compiler as comp_mod

    monkeypatch.setattr(comp_mod, "_probe_encoder_available", lambda enc: True)
    comp = VideoCompiler(str(tmp_path / "out"), encoder="h264_nvenc")
    args, chosen = comp._select_encoder()
    assert chosen == "h264_nvenc"
    assert args[:2] == ["-c:v", "h264_nvenc"]
    assert "-cq" in args # NVENC constant-quality knob (not -crf)

def test_select_encoder_falls_back_to_libx264_when_nvenc_unavailable(tmp_path, monkeypatch):
    """On a box where NVENC can't actually run (CI / no GPU), the requested encoder falls back
    to
    libx264 so a misconfig never nukes the reel."""
    from backend.pipeline.stitcher import compiler as comp_mod

    monkeypatch.setattr(comp_mod, "_probe_encoder_available", lambda enc: False)
    comp = VideoCompiler(str(tmp_path / "out"), encoder="h264_nvenc")
    args, chosen = comp._select_encoder()
    assert chosen == "libx264"
    assert args == ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"]

def test_compile_uses_selected_encoder_in_trim(tmp_path, monkeypatch):
    """End-to-end (ffmpeg mocked): with an available NVENC encoder, every per-clip trim command
    carries -c:v h264_nvenc, and the concat stays -c copy."""

```

42. user (2026-07-08T06:13:18.804Z)

```

=== test_compiler.py tail (>=200) ===
carries -c:v h264_nvenc, and the concat stays -c copy."""
from backend.pipeline.stitcher import compiler as comp_mod

monkeypatch.setattr(comp_mod, "_probe_encoder_available", lambda enc: True)
comp = VideoCompiler(str(tmp_path / "out"), encoder="h264_nvenc")
raw = tmp_path / "raw.mp4"

```



```

raw.write_bytes(b"x")
trims = []

def fake_run(cmd, *a, **k):
    if cmd[0] == "ffprobe":
        return MagicMock(stdout="100.0\n", returncode=0)
    out = cmd[-1]
    if isinstance(out, str) and out.endswith(".mp4"):
        open(out, "wb").write(b"clip")
        if "-i" in cmd and cmd[cmd.index("-i") + 1] == str(raw):
            trims.append(cmd)
    return MagicMock(returncode=0, stdout=b"", stderr=b"")

with patch.object(subprocess, "run", side_effect=fake_run):
    comp.compile_highlights(str(raw), [(0.0, 5.0), (10.0, 15.0)], "reel.mp4")
assert trims, "no trim commands captured"
for cmd in trims:
    assert "h264_nvenc" in cmd # hardware encode on every clip
    assert "libx264" not in cmd
=== total lines ===
224

```

43. assistant (2026-07-08T06:13:50.948Z)

Let me read the worker `cmd\_compile` tests and the compile-dispatch tests to confirm coverage gaps.

44. user (2026-07-08T06:13:54.042Z)

```

=== test_worker.py compile tests (dac6736) ===
30: def test_coerce(raw, expected):
34: def test_apply_overrides_on_typed_config():
49: def test_apply_overrides_rejects_bad_format():
55: def test_write_intervals_csv(tmp_path):
73: def test_write_report_json(tmp_path):
83: def test_parser_infer_accepts_set_and_uris():
101: def test_parser_infer_skip_validation_flag():
109: # ----- cmd_infer (local, mocked)
169: def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
227: def test_setup_logging_levels():
236: def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path, monkeypatch, caplog):
279: def test_cmd_infer_detector_failure_marks_report_and_marker_failed(
283:     as a 0-segment "success". cmd_infer returns rc 3 and writes report.json + the done-
marker with
319:     # ...and the completion marker the cloud dispatcher bucket-polls carries the same
verdict.
320:     marker = json.loads(done.read_text())
321:     assert marker["returncode"] == 3 and marker["status"] == "failed"
322:     assert marker["segment_count"] == 0 and marker["video_id"] == "vidx"
325: def test_cmd_infer_real_trajectory_segmenter_failure_flows_to_report(tmp_path, monkeypatch):
327:     through cmd_infer -> _fail(3) -> a status="failed" report.json + done-marker. Locks the
two
328:     halves (segmenter Failure + worker report/marker) TOGETHER, not just via the _FailSeg
stub. """
377:     marker = json.loads(done.read_text())
378:     assert marker["returncode"] == 3 and marker["status"] == "failed"
381: def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
412:     # success): report.json status="failed" + the done-marker rc 2 / status="failed".
415:     marker = json.loads(done.read_text())
416:     assert marker["returncode"] == 2 and marker["status"] == "failed"
419: def test_cmd_infer_skip_validation_bypasses_validators(tmp_path, monkeypatch):
479: def test_label_clip_name_strips_date_prefix(fname, expected):
484: def test_cmd_train_detector_joins_dated_labels_to_bare_videos(tmp_path, monkeypatch):
555: def test_cmd_train_detector_skips_label_with_no_matching_video(tmp_path, monkeypatch):
623: def test_cmd_train_detector_skips_unprobeable_and_trajless_videos(

```

```

707: def test_cmd_train_detector_aborts_clean_when_fewer_than_two_videos_join(
758: def test_cmd_train_orchestration(tmp_path, monkeypatch):
812: def test_parser_train_detector_args():
831: def test_cmd_train_detector_source_runs_real_extract_via_stub(tmp_path, monkeypatch):
903: def test_cmd_train_detector_native_on_tracknet_fails_clean_not_traceback(
942: def test_probe_video_fps_width_returns_none_on_unreadable(tmp_path):
952: def test_parser_compile_accepts_spec_and_done_uris():
963:     "storage.output_root=gs://b",
968:     assert args.done_uri == "gs://b/_compile/tok.done"
969:     assert args.set == ["storage.output_root=gs://b"]
977:     "locale": None,
989: def test_cmd_compile_stitches_and_writes_marker(tmp_path, monkeypatch):
991:     no ffmpeg), mirrors the reel, and writes a compile done-marker for the dispatcher's
bucket-poll.""
1003:     def _fake_stitch(cfg, stitching, video_id, source_ref, time_pairs, out_name, locale,
notify):
1012:         "reel_uri": f"gs://b/highlights/{video_id}/{out_name}",
1018:         cfg.write_text('{"storage": {"backend": "local"}}')
1034:         f"storage.output_root={bucket}",
1043:         marker = json.loads(open(done, encoding="utf-8").read())
1044:         assert marker["returncode"] == 0 and marker["status"] == "success"
1045:         assert marker["download_url"] == "/api/highlights/vidReel/reel.mp4"
1048: def test_cmd_compile_failure_returns_rc3_and_failed_marker(tmp_path, monkeypatch):
1049:     """"A stitch failure surfaces as rc 3 + a failed marker (rc != 0) so the dispatcher's
poll
1066:     cfg.write_text('{"storage": {"backend": "local"}}')
1085:     marker = json.loads(open(done, encoding="utf-8").read())
1086:     assert marker["returncode"] == 3 and marker["status"] == "failed"
1089: def test_cmd_compile_no_done_uri_writes_no_marker(tmp_path, monkeypatch):
1090:     """"DEFAULT-OFF: a direct-CLI stitch with no --done-uri writes no marker.""
1103:     cfg.write_text('{"storage": {"backend": "local"}}')

```

45. user (2026-07-08T06:14:02.431Z)

```

195         )
196         return bool(path) and os.path.exists(os.path.expanduser(path))
197
198
199     def _cloud_available(cfg) -> bool:
200         """"Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).
201
202         True when the control plane is wired for Cloud Run: either the configured default
target is
203         already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
204         (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)
205         are present so a forced per-request override can actually dispatch + collect a
result.
206
207         Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
would work.""
208         deployment = getattr(cfg, "deployment", None)
209         if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
210             return True
211         has_job = bool(
212             os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
213         )
214         out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
215         return bool(has_job and out_root)
216
217
218     def _will_detect_on_cloud(cfg, req_compute: Optional[str]) -> bool:
219         """"Would DETECT for this request run on the L4 worker (vs in-process)? Mirrors the

```

```

220         ``_maybe_dispatch_remote`` decision WITHOUT side effects (#521), so
``_execute_processing`` can
221         decide whether to DELEGATE validation to the worker
(``phase_placement.validation``). A per-request
222         picker choice wins; otherwise ``phase_placement.segment`` (whose legacy default is
cloud iff
223         ``compute_target==cloud_run``) gates it, and cloud must actually be available. A
pre-dispatch reject
224         (e.g. cloud requested but unconfigured) counts as NOT-on-cloud ? nothing runs
remotely. """
225         choice = (req_compute or "").strip().lower()
226         if choice == "local":
227             return False
228         if choice == "cloud":
229             return _cloud_available(cfg)
230         return (
231             resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
232             and _cloud_available(cfg)
233         )
234

```

46. assistant (2026-07-08T06:14:23.234Z)

Confirmed: `\_cloud\_available` returns True for `compute\_target=="cloud\_run"` without checking that `output\_root` is a URI. Let me read the compile-dispatch tests and the worker `cmd\_compile` full body to finalize.

47. user (2026-07-08T06:14:26.777Z)

```

=== test_compile_dispatch.py (dac6736) function list + key asserts ===
44:         status = "success" if self.returncode == 0 else "failed"
48:         "status": status,
49:         "download_url": f"/api/highlights/{vid}/{out_name}",
50:         "reel_uri": f"gs://bucket/highlights/{vid}/{out_name}",
91:         "storage": {"backend": "local", "output_root": str(bucket)},
108: def test_compile_dispatches_to_l4_and_returns_reel(compile_env):
115:     assert out["status"] == "success"
116:     assert out["video_id"] == video_id
117:     assert out["download_url"] == f"/api/highlights/{video_id}/myreel.mp4"
118:     assert out["compute"]["path"] == "cloud_run"
119:     assert out["compute"]["execution"] == "exec-compile-1"
120:     assert len(fake.calls) == 1 # exactly one Cloud Run Job execution
123: def test_compile_dispatch_stages_resolved_spec_with_nvenc(compile_env):
133:     assert fake.spec is not None
134:     assert fake.spec["video_id"] == video_id
135:     assert fake.spec["video_uri"].endswith("source.mp4")
137:     assert fake.spec["time_pairs"] == [[2.0, 6.0], [10.0, 14.0], [20.0, 22.0]]
139:     assert fake.spec["stitching"]["encoder"] == "h264_nvenc"
142:     assert argv[0] == "compile"
143:     assert "--spec-uri" in argv and "--done-uri" in argv
145:     assert "storage.output_root=" in joined # bucket forwarded so the worker mirrors the
reel
148: def test_compile_dispatch_forwards_stitch_config_overrides(compile_env):
150:     (storage.output_root/backend) ? else the worker can't read the spec or mirror the
reel. """
156:     assert f"storage.output_root={bucket}" in joined
157:     assert "storage.backend=local" in joined
160: def test_compile_cloud_failure_returns_failed(compile_env):
168:     assert out["status"] == "failed"
169:     assert "stitch blew up" in out["message"]
170:     assert out["video_id"] == video_id
173: def test_compile_default_stitches_in_process_no_dispatch(compile_env, monkeypatch):
186:     return {"status": "success", "video_id": vid, "download_url":
f"/api/highlights/{vid}/{name}"}
191:     assert out["status"] == "success"

```

```

192:     assert fake.calls == [] # NEVER dispatched ? the control-plane stitched in-process
193:     assert seen["pairs"] == [(2.0, 6.0), (10.0, 14.0), (20.0, 22.0)]
194:     assert seen["source"].endswith("source.mp4")
197: def test_compile_missing_video_fails_before_dispatch(compile_env):
203:     assert out["status"] == "failed"
204:     assert fake.calls == [] # never dispatched ? resolve failed first

```

48. assistant (2026-07-08T06:15:11.506Z)

Let me pin exact line numbers in the merged tree for the findings.

49. user (2026-07-08T06:15:17.148Z)

```

=== orchestration.py anchors ===
615:     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
1172:     reel_uri = reel_object_uri(cfg, video_id, name)
1190: def _stitch_reel(
1256:     reel_uri = reel_object_uri(cfg, video_id, out_name)
1279: def _dispatch_compile_remote(
1307:     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
1308:     compute = get_compute_backend(cfg, target_override="cloud_run") if out_root else None
=== compiler.py probe anchors ===
31: def _probe_encoder_available(encoder: str) -> bool:
52:     "-frames:v",
54:     "-c:v",
277:     def _select_encoder(self) -> Tuple[List[str], str]:
278:         """Resolve the ``-c:v ?`` args used for EVERY per-clip trim re-encode this run, plus
the
293:         return ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"], enc
294:         # NVENC (H.264/HEVC): hardware encode. -cq is the CRF-equivalent constant-quality
knob; p4 is
297:         ["-c:v", enc, "-preset", "p4", "-cq", "23", "-pix_fmt", "yuv420p"],
379:         else ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"]
=== worker cmd_compile output_root fallback ===
262:     config.output.output_dir = scratch
464: def cmd_compile(args: argparse.Namespace) -> int:
481:     config.output.output_dir = scratch
483:     # fallback so reel_object_uri (which reads storage.output_root) always resolves the
reel's bucket
486:     config.storage.output_root = args.out_uri

```

50. assistant (2026-07-08T06:15:42.048Z)

Let me verify `get\_compute\_backend` doesn't inject a fixed token in production (which would break concurrent-dispatch marker uniqueness).

51. user (2026-07-08T06:15:45.970Z)

```

=== get_compute_backend / CloudRunCompute construction (dac6736) ===
44: def get_compute_backend(
49:     token: Optional[str] = None,
55:     Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
73:     from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
91:     return CloudRunCompute(
98:         token=token,
--- token= usages across backend ---
49:     token: Optional[str] = None,
98:         token=token,

```

52. assistant (2026-07-08T06:16:05.716Z)

I have completed a thorough parity-concurrency review of the #521/#525 diff (merged commit `dac6736`, reviewed against its parent `bf97700` ? note the working tree on disk carries

unrelated uncommitted `rendition` WIP, so I reviewed the committed `dac6736` tree, not the disk files).

Verified-sound invariants: one encoder resolved once per reel via `\_select\_encoder()` and threaded to every `\_trim\_one\_segment` (concat stays `-c copy`); NVENC probe memo `\_ENCODER\_AVAILABLE` is correct and benign under the GIL (compile is single-threaded per process / serialized by `\_LOCAL\_COMPUTE\_LANE`); marker prefixes are distinct (`\_dispatch/` for infer, `\_compile/` for compile); token uniqueness holds (`get\_compute\_backend` builds a fresh `CloudRunCompute` with `token=None` per dispatch ? per-call `uuid4`); the spec is `storage.put` (synchronous, strongly-consistent) before `run\_job`, so no staging/dispatch race; and validation delegation (`\_will\_detect\_on\_cloud` vs `\_maybe\_dispatch\_remote` vs `\_execute\_processing`) is consistent so a clip is validated exactly once in every branch.

Two real defects found.

53. user (2026-07-08T06:16:23.078Z)

Structured output provided successfully